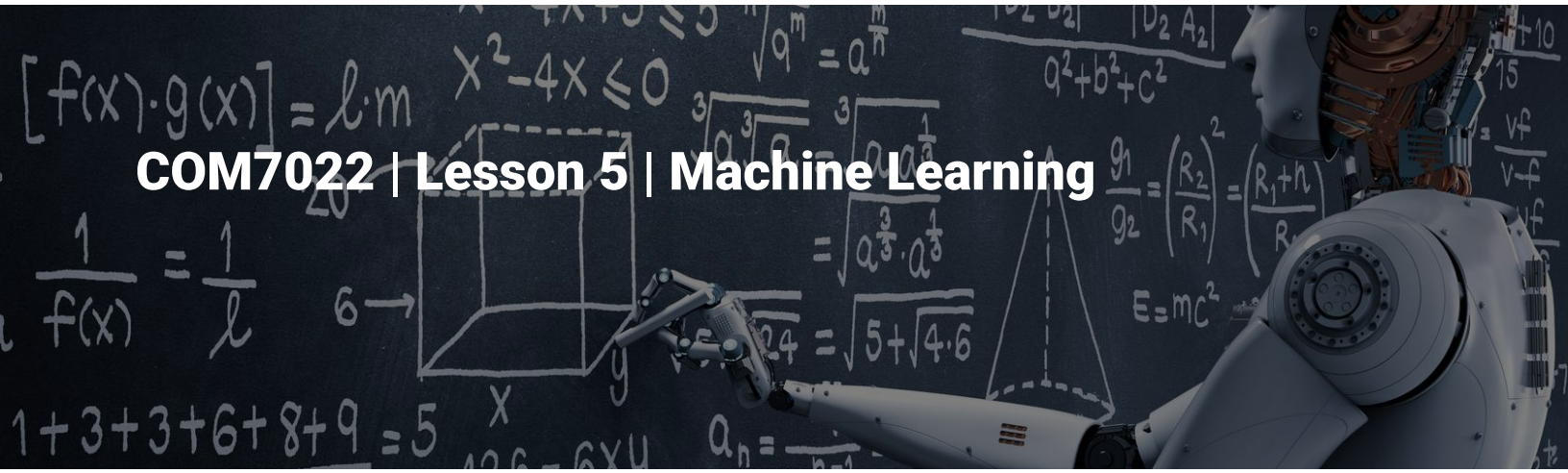


COM7022 | Lesson 5 | Machine Learning

Machine Learning (COM7022)

Lesson 5: Supervised Learning – Decision Trees

Cover image by Phonlamai Photo from Shutterstock.com

- **Lesson introduction**
- **Before you get started**
- **Topic 1: Decision trees**
- **Topic 2: Gain ratio**
- **Topic 3: Random forest**
- **Topic 4: Pruning**
- **Knowledge check**

Lesson summary

Reading List

Lesson introduction

Welcome to Lesson 5

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Welcome to Lesson 5!

Lesson 5 has been designed to take you through the complexities of supervised machine learning. Having already discussed association in the previous lesson, we are in a position where we can begin to move onto further analytical methodologies.

Classification is a methodology that is widespread not only within machine learning, but also across data science in its entirety. As you move through your professional career, you will no doubt engage in one, if not all, the common topics detailed within this lesson.

Computing
At Arden University

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

By the end of this week's lesson, you will:

- Know what a decision tree model is and how various process can be applied to improve accuracy.
- Understand what gain ratio is and how it can be applied as an alternative to information gain.
- Appreciate the progression from singular classification trees to random forest classification.
- Fully understand the fundamental principles of decision tree pruning.

The module's learning objectives are:

- Demonstrate a practical understanding of the characteristics of various machine-learning methods and their applications in data analysis.
- Select and apply appropriate machine-learning techniques for a given scenario.
- Critically evaluate the results from the application of machine-learning techniques and propose improvements.

Graduate Attribute: Professionally knowledgeable in their subject area.

- Critically evaluate and further contextualise understanding of the subject area with the ability to link the discipline(s) to local, national and global issues, research and pursue evidence-based arguments within the discipline using an extensive range of academic and professional body resources. Show clear ability to engage with different traditions of thought and the ability to apply their knowledge in practice, including in multidisciplinary or multi-professional contexts.
- Lesson 5 aligns with Learning Objective 1: **Demonstrate a practical understanding of the characteristics of various machine-learning methods and their applications in data analysis**.

Computing

At Arden University

We have worked across teams at Arden University to try and create an accessible and inclusive experience within the learning content. We are continuously assessing the accessibility of our lessons to make improvements and enhancements based on testing and feedback.

This lesson may include screenshots of code and software used. Please be aware that these may not always have a text alternative, and that other features such as size and colour may not be accessible.

If you experience any issues with accessing the learning content throughout this module, there are several support services in place to help you.

- Arden University has a dedicated Inclusion Services Team who can help you with getting adjustments and equipment.
- The Library Portal also has information on the accessibility tools that can help you access resources: Accessibility Tools – Library Portal
- If you are struggling with accessing a specific part of the content e.g. a video resource that is missing captions, please speak to your Lecturer who may be able to advise on an alternative resource in the interim to ensure you are able to get the learning you need to progress with the lesson.

Continue to the Before you get started section

Before you get started

Image from Articulate Rise 360 Content Library.

Did you know?

Machine learning is a required skill for over 16,000 jobs in the UK listed on LinkedIn (LinkedIn, 2023).

What do you think the average salary for a computer scientist in the UK is?

- ◯ £30,000
- ◯ £40,000
- ◯ £50,000

SUBMIT

Continue

The average salary for a computer scientist in the UK is £50,000 (Talent.com, 2024).

Continue to Topic 1

Topic 1: Decision trees

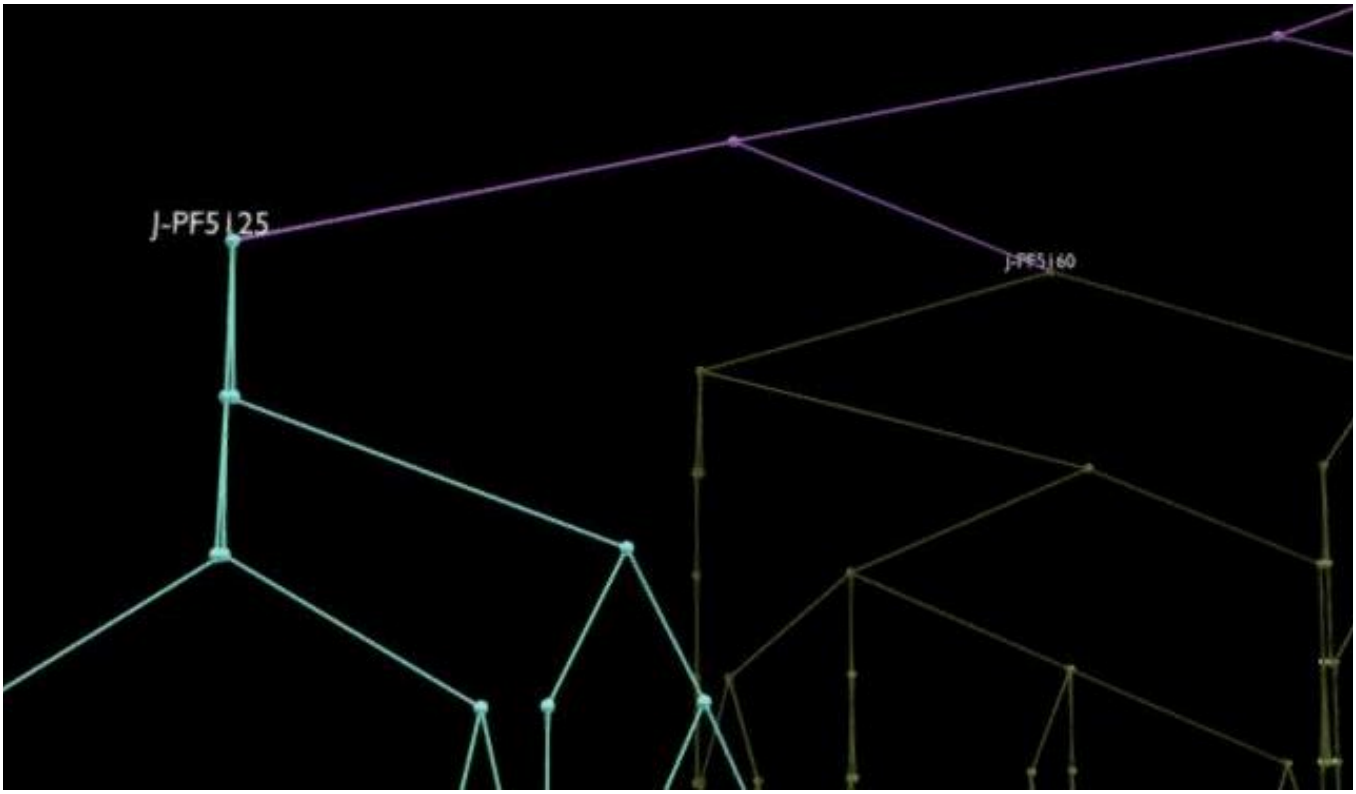


Image by DaveVance from Sketchfab.com.

The decision tree is arguably the most popular tool for classification within data mining. It is called a decision tree because it is, in essence, a flowchart that resembles the structure of a tree, where each internal node denotes a test on an attribute. Each branch of the decision tree represents an outcome of the test and each leaf node (terminal node) holds a class label.

A tree can split the dataset into subsets, based on an attribute value. The process is repeated on a subset in a recursive manner, called recursive partitioning. The recursion is completed when the subset at a node (leaf) has the same value as the target variable, or when splitting no longer adds value to the predictions. An important advantage of a decision tree classifier is that it does not require any domain knowledge, or parameter setting and, as such, it is ideal for knowledge discovery. The schema of a decision tree can be seen below in Figure 5.01.

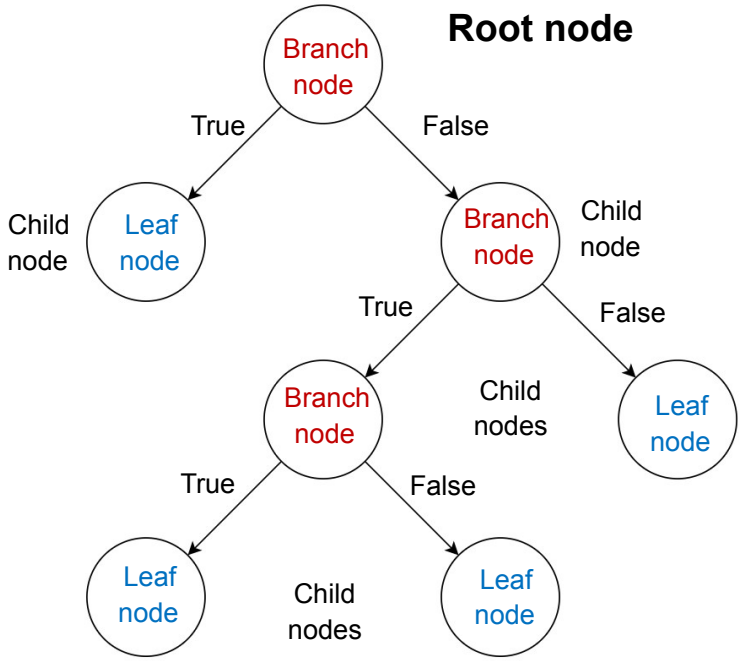


Figure 5.01. An arbitrary decision tree schema by Arden University. (Accessible text). (Accessible list).

As you have no doubt guessed already, a decision tree makes a decision on whether or not the statement is true or false, and the statements are contained within the branch nodes. It is important to note at this early stage that when a decision tree classifies things into categories, it is known as a classification tree. However, when a decision tree predicts numerical values, it is known as a regression tree. Moreover, when engaging with decision trees, it is fine to mix data types (i.e., numeric and Boolean). One final note is that there can be multiple numeric thresholds within the same tree (e.g., <30*lbs*,\ >30*lbs*\ and\ <15*lbs*). In this topic, we will only be concentrating

on a classification tree. The regression tree will not be covered, but it has been briefly mentioned here for completeness.

Classification trees are pretty intuitive and easy to work with. As Figure 5.01 demonstrates, you start at the top, or the root node, and then work your way down the branches until you get to a point where you cannot go any further, thus resulting in a classification. Also contained within Figure 5.01, upon the arrows, are true and false: if something is true, then it takes the left arrow; if something is false, the decision goes to the right. Now that we know this, for the remainder of this topic we will leave out the true/false conventions. Now let us run through an example in a step-by-step manner, using fictitious data, before we give a coded example of the algorithm.

Continue

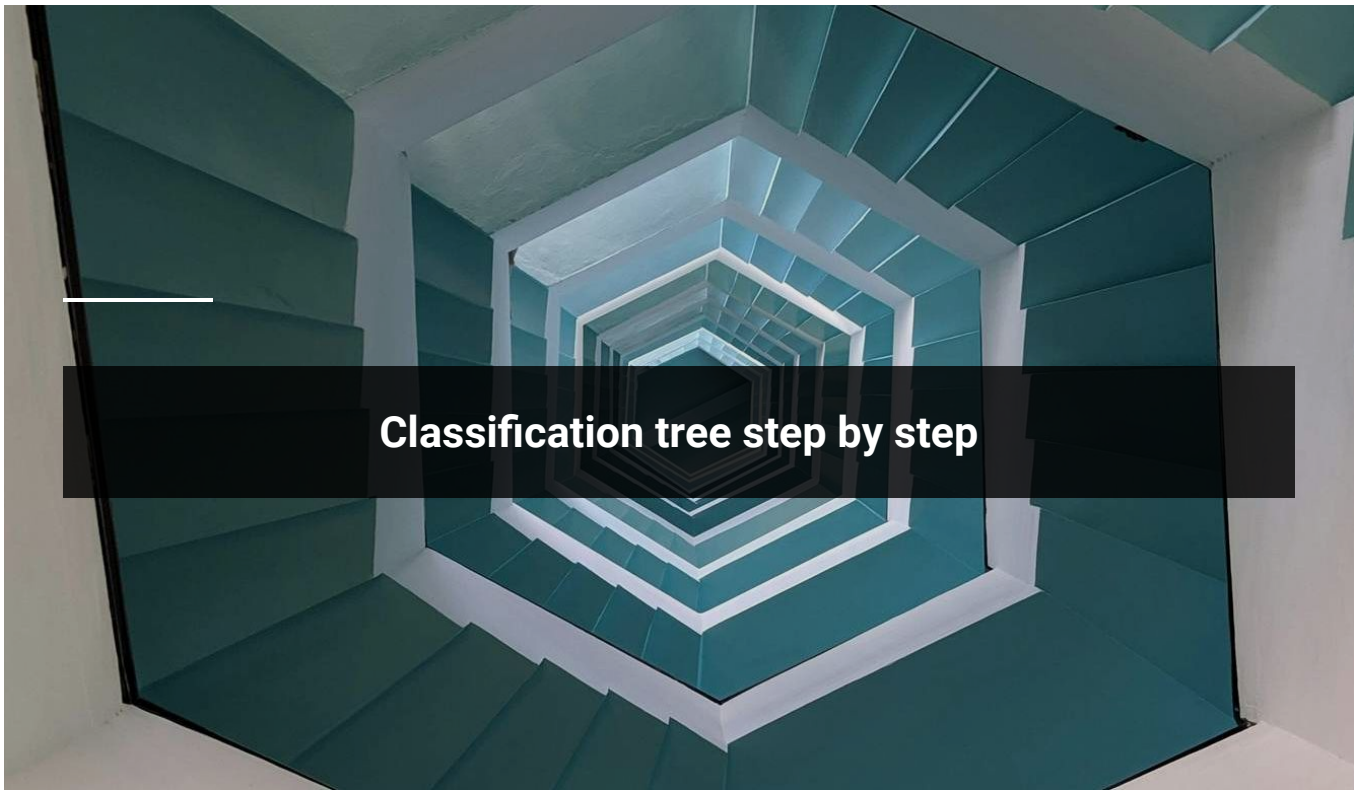


Image from Articulate Rise 360 Content Library.

The data that we will use to run through this example of building a classification tree is below:

Likes Nachos	Likes Sprite	Age	Likes the film "Star Wars"
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	Yes	83	Yes

Table 5.01.

The first question and starting point we need to determine is: *What variable should be at the top of the tree so that we get the best results possible?* To answer this question in the most honest way possible, to begin with, we begin with a pure guess. As such, we will use the guess “Likes Nachos” variable to predict whether a person likes Star Wars or not. This is demonstrated in Figure 5.02 below.

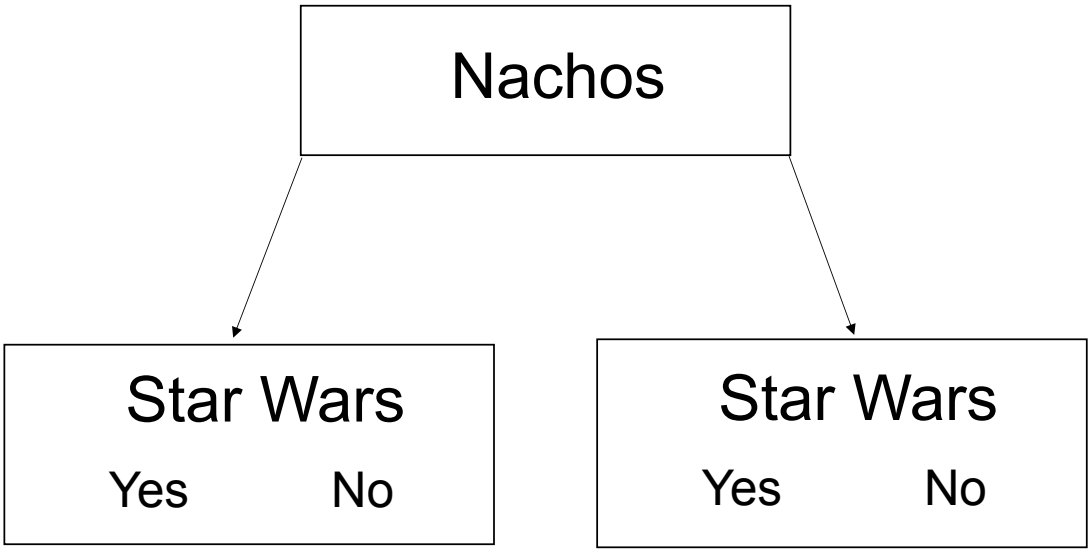


Figure 5.02. Initial stages of our classification tree on whether someone likes Nachos and likes Star Wars by Arden University. (Accessible text). (Accessible list).

Likes Nachos	Likes the film "Star Wars"
Yes	No
Yes	No
No	Yes
No	Yes
Yes	Yes
Yes	No
No	Yes

Table 5.02

We now work our way down the data set using Table 5.02 above and input the relevant information into the tree. For example, we can clearly see that the first person likes Nachos, so we automatically use the left-hand branch, as it is true. However, the person does not like the film Star Wars. Therefore, we input this data into the left-hand branch also. This results in the tree seen in Figure 5.03.

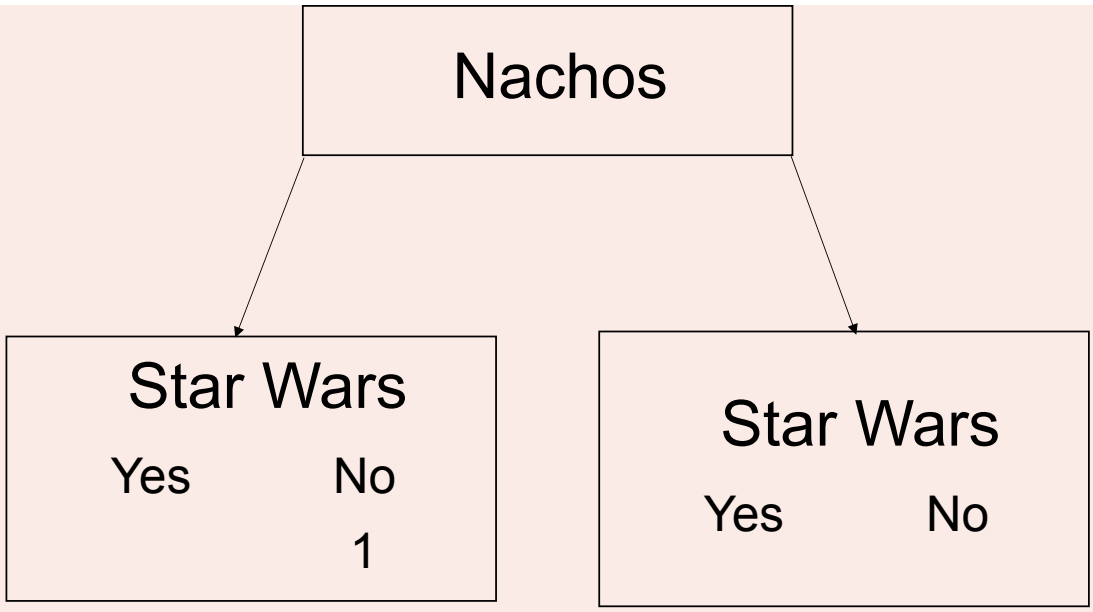


Figure 5.03. Demonstrating the initial data point being inserted into the tree, based on Table 5.02 by Arden University. (Accessible text). (Accessible list).

The second person also like Nachos, so again we go down the left-hand branch; as with the first person, they too do not like Star Wars. Figure 5.04 demonstrates this.

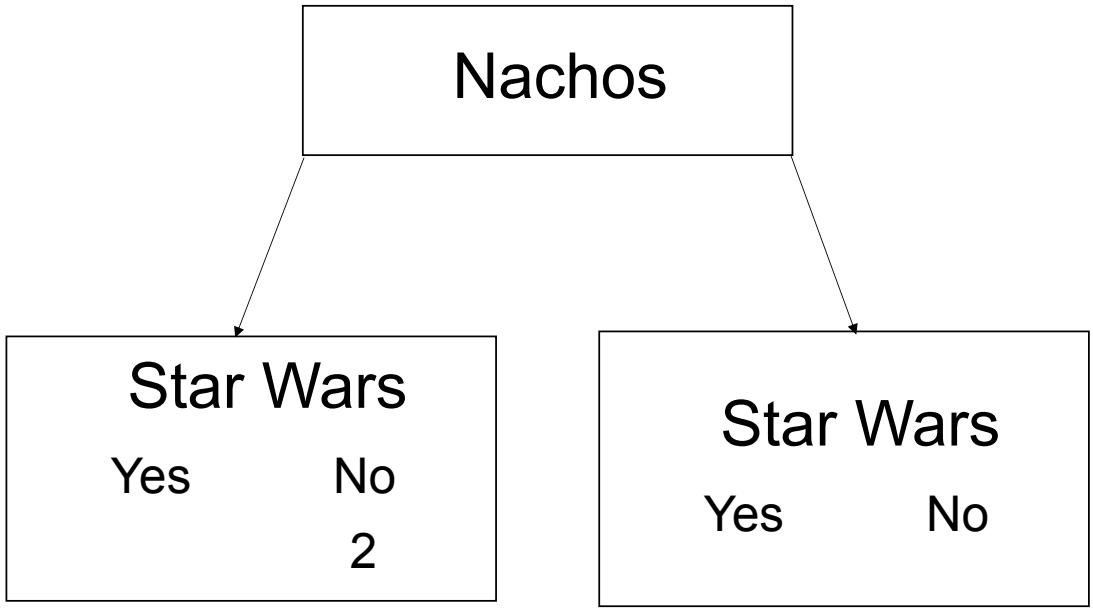


Figure 5.04. Resulting tree after the second data point has been included by Arden University. (Accessible text). (Accessible list).

The third person is different: they do not like Nachos, so this time we go down the right-hand branch. However, this person does like Star Wars, so the tree after the third person's data has been inserted can be seen in Figure 5.05.

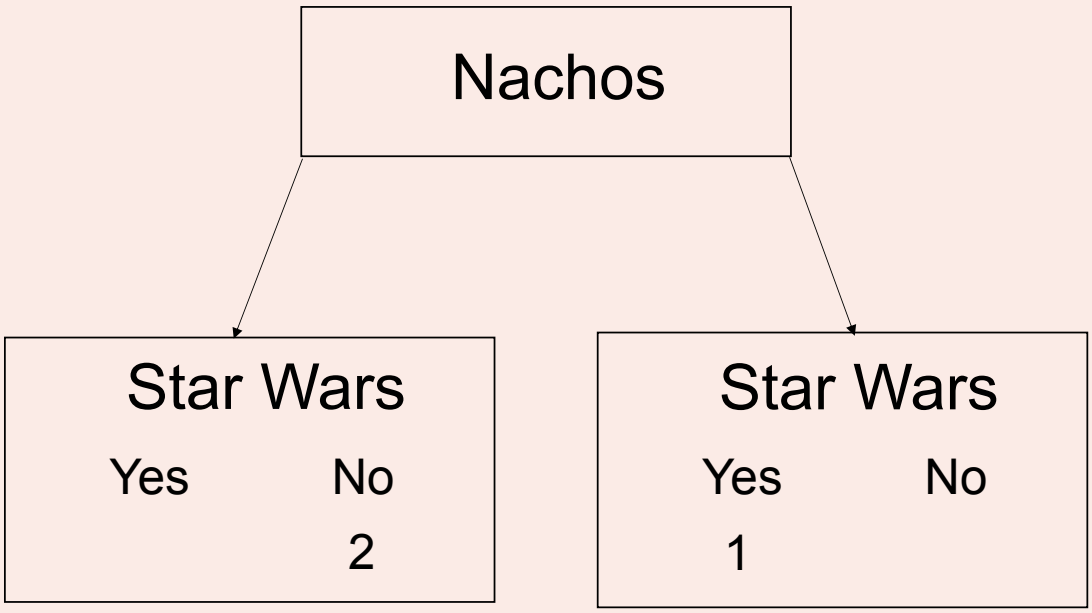


Figure 5.05. The classification tree after the third person's data has been included by Arden University. (Accessible text). (Accessible list).

If we repeat the process for the remaining people, we end up with the final classification tree shown in Figure 5.06.

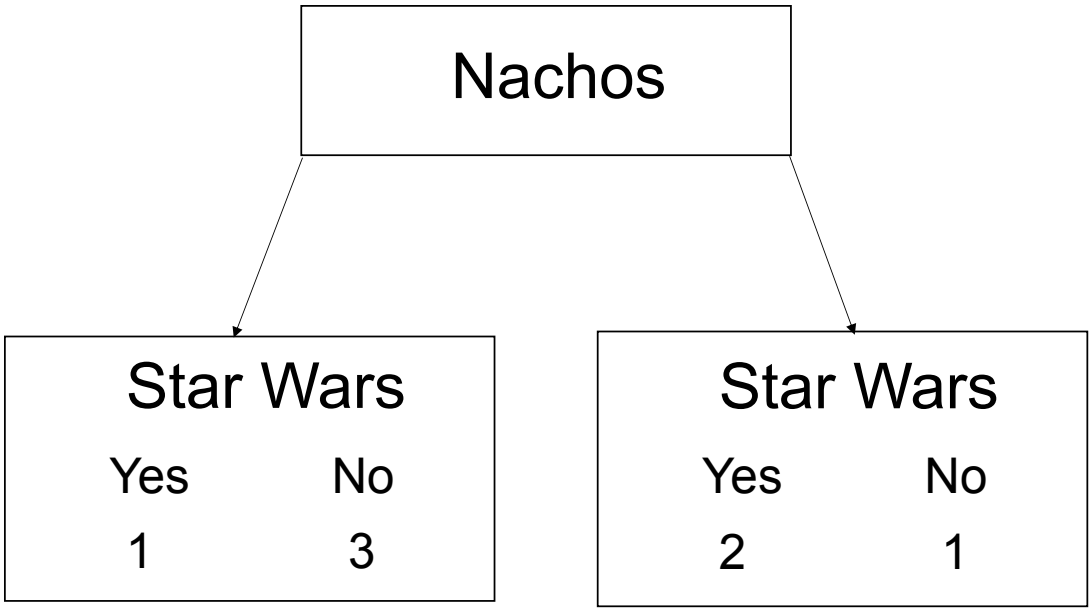


Figure 5.06. Completed classification tree on whether someone likes Nachos and likes Star Wars by Arden University. (Accessible text). (Accessible list).

We can do the exact same process for the “like Sprite” variable. The resulting classification tree using this variable is shown in Figure 5.07.

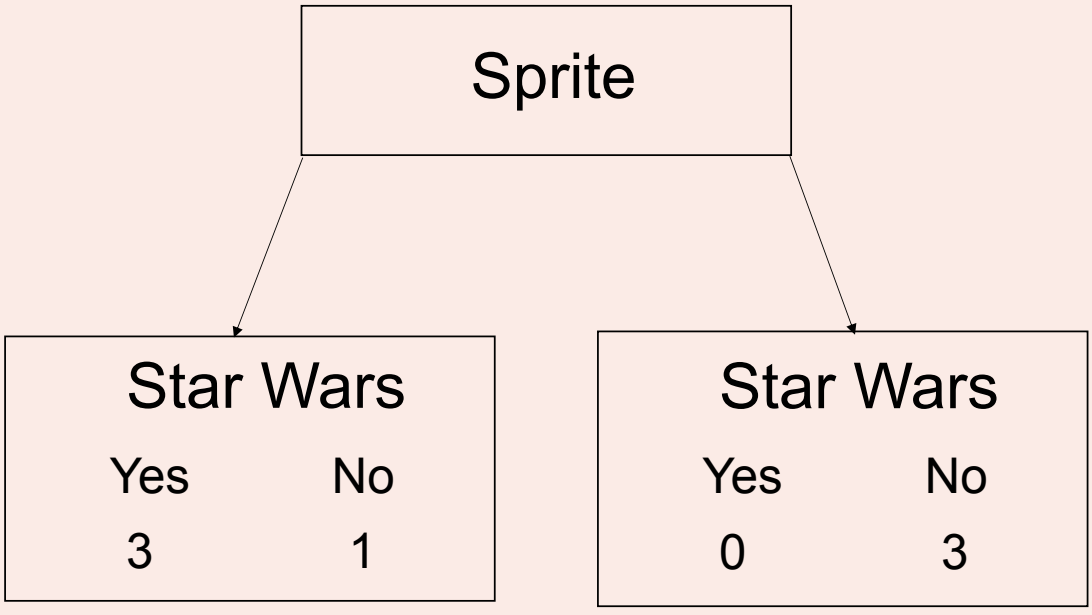


Figure 5.07. The completed classification tree using the variable Sprite by Arden University. (Accessible text). (Accessible list).

By comparing the Nachos and Sprite trees, we can determine that neither one does well at predicting who will and who will not like Star Wars. Specifically, the leaves below contain a mixture of who does and who does not like Star Wars. When we have a mixture like this, the leaves are called ***impure leaves***.

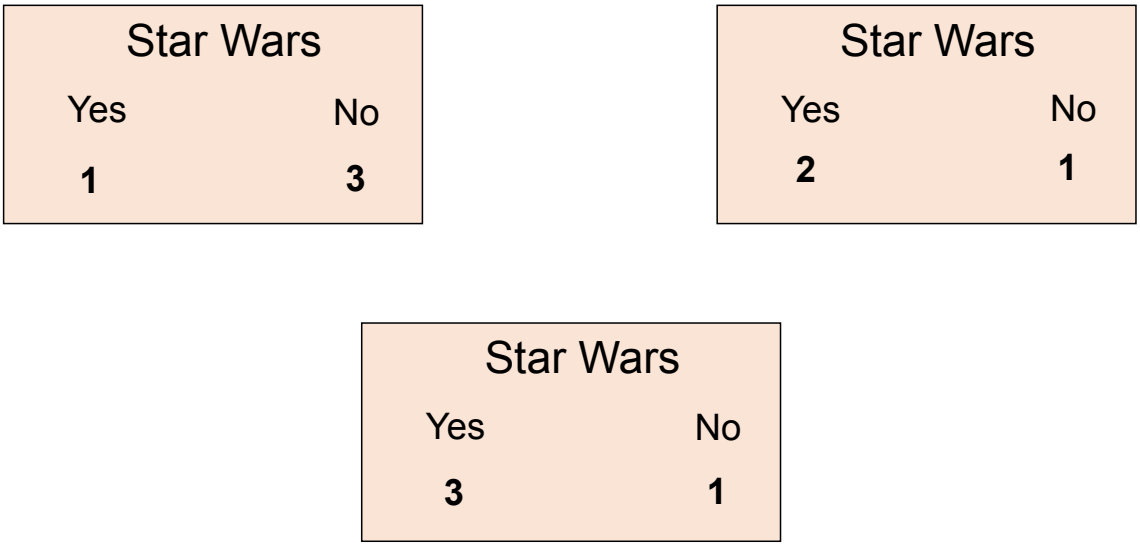


Image by Arden University. (Accessible text). (Accessible list).

On the other side, there is one leaf that only contains people who do not like Star Wars. This is called a ***pure leaf*** and is shown below.

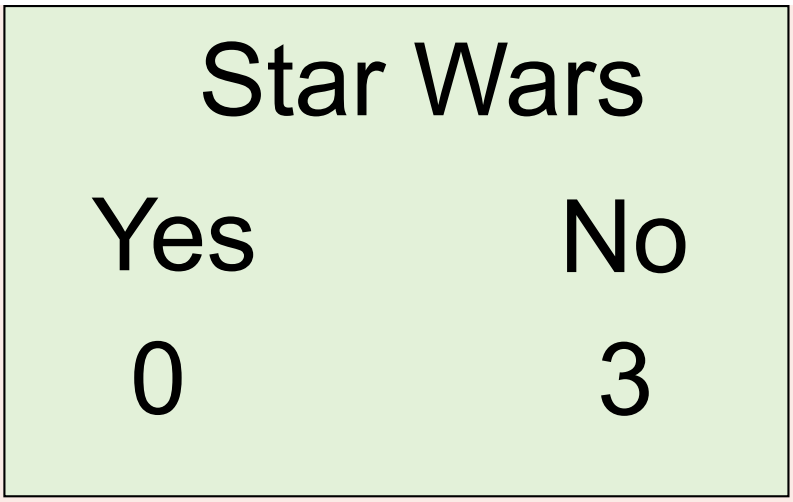


Image by Arden University. (Accessible text). (Accessible list).

As leaves in the Nachos tree are impure and only one leaf in the Sprite tree is impure, we would naturally assume that the Sprite tree does a much better job at predicting future data of who does and who does not like Star Wars. However, this would just be speculation on our part. It would be better if we could quantify the differences between the Nachos and Sprite trees. In fact, there are several methods which can be applied to quantify the data, such as Gini impurity (GI), entropy and information gain. As GI is the most popular and straightforward method in quantifying decision trees, we will be using it in this topic. To calculate the GI from the Nachos tree, we need to focus on the separate leaves and determine the GI for each, then compare overall. Let's get the Nachos tree here again so that the calculations are easier to see, as shown in Figure 5.08.

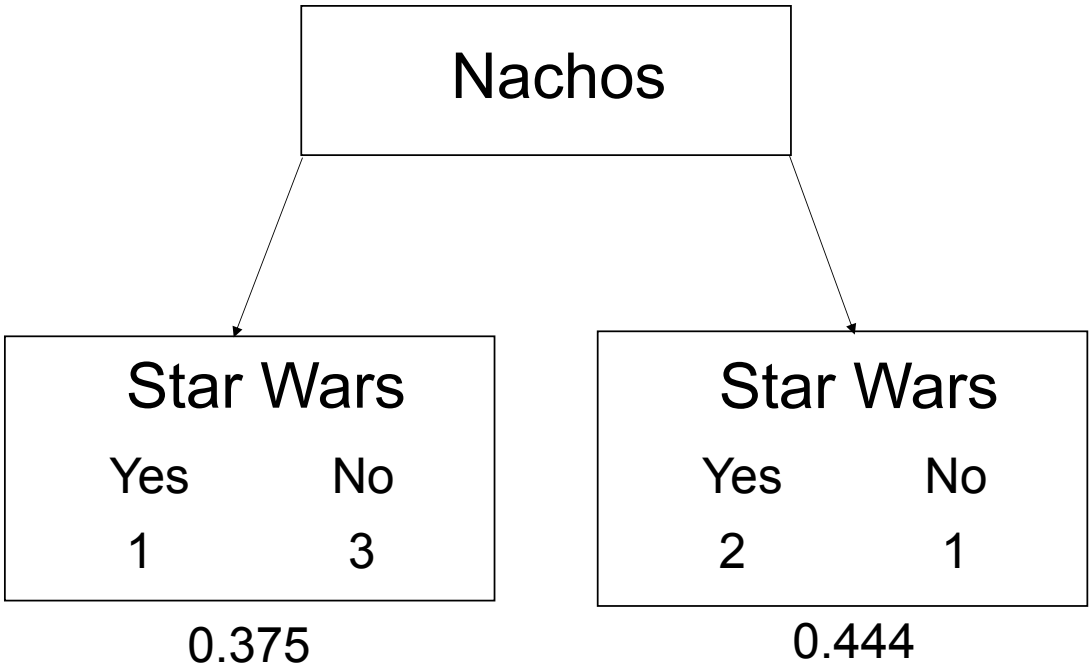


Figure 5.08. Figure by Arden University. (Accessible text). (Accessible list).

The calculation for the GI on the left-hand leaf is:

$$GI = 1 - (the\ probability\ of\ "Yes")^2 - (the\ probability\ of\ "No")^2$$

$$= 1 - (\frac{1}{1+3})^2 - (\frac{3}{1+3})^2 = \mathbf{0.375}$$

We derive the GI for the leaf on the right is in exactly the same manner:

$$= 1 - (\frac{2}{2+1})^2 - (\frac{1}{2+1})^2 = \mathbf{0.444}$$

Notice that the leaves do not represent the same number of people. Therefore, the total GI is the weighted average of the leaf impurities. To calculate this, the weight for the leaf on the left is the total number of people within the leaf, divided by the total number of people in both leaves. We

then multiply this by its associated GI. Finally, we do the same calculations to the leaf on the right and sum together to get the actual GI for Nachos. Mathematically, this is shown below.

$$= \left(\frac{4}{4+3}\right) * 0.375 + \left(\frac{3}{4+3}\right) * 0.444 = \mathbf{0.405}$$

If we follow the exact same methodology for Sprite, we have an actual GI of **0.214**. The final variable that we need to determine is Age. Age however contains numerical values, which makes determining the GI for this variable slightly more involved – but nothing you cannot handle. The first step is to sort the numerical values from the lowest to the highest value. The next step is to determine the average age for all adjacent ages. This will result in Table 5.03 below.

9.5	Age
15	7
26.5	12
36.5	18
44	35
66.5	38
	50
	83

Table 5.03. (Accessible table).

We then determine the GI values for each average age, resulting in Table 5.04 below.

9.5	Age	0.429
15	7	0.343
26.5	12	0.476
36.5	18	0.476
44	35	0.343
66.5	38	0.429
	50	
	83	

Table 5.04. (Accessible table).

From this derivation, we can construct our tree with Age < 9.5 as the root. As there is only one person with Age < 9.5 who does not like Star Wars, they are placed within the left leaf, while everyone else is placed in the right leaf, as shown below in Figure 5.09.

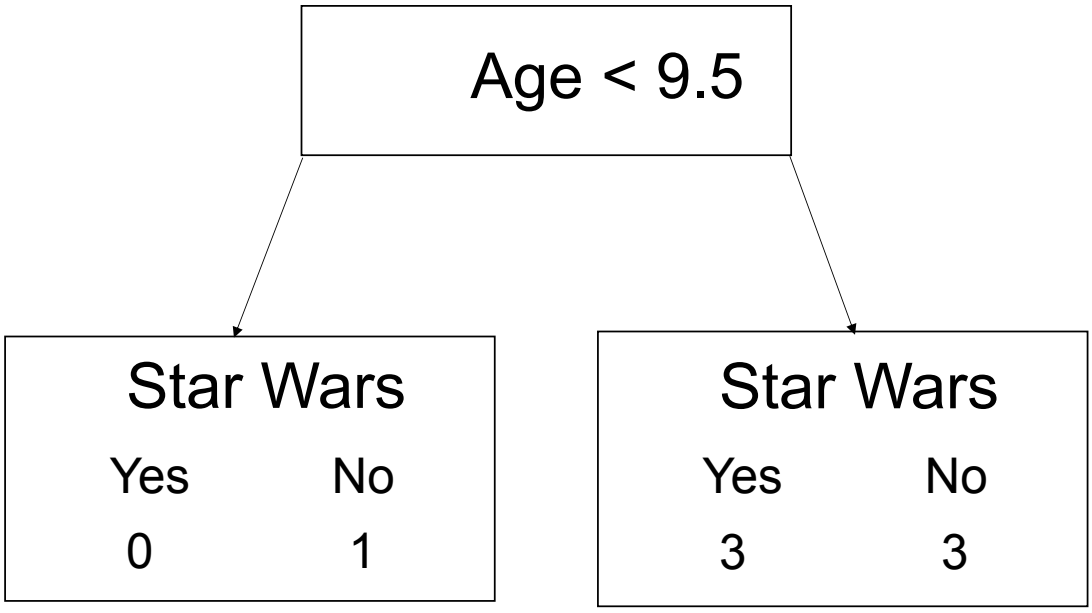


Figure 5.09. The resulting classification tree using Age < 9.5 as the variable. Image by Arden University. (Accessible text) (Accessible list).

Now calculate the GI once more on the leaf on the left. You will find that this results in 0 (zero). However, if you consider this result for a second, it makes perfect sense, as every person in this leaf does not like Star Wars, hence no impurity. The mathematics are shown below:

$$= 1 - (\frac{0}{0+1})^2 - (\frac{1}{0+1})^2 = \mathbf{0}$$

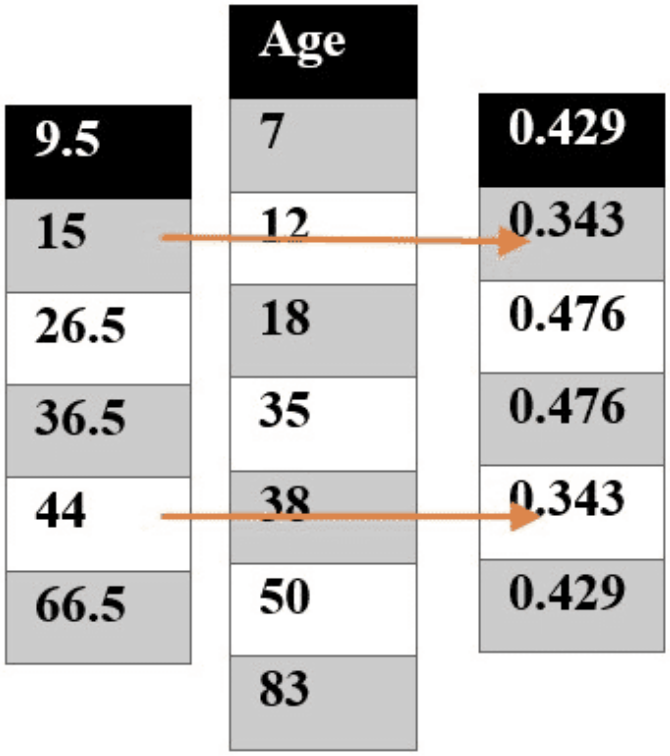
We then do the same for the right leaf and determine that the GI for that leaf, which results in 0.5. Again, the mathematics are shown below:

$$= 1 - (\frac{3}{3+3})^2 - (\frac{3}{3+3})^2 = \mathbf{0.5}$$

Now, as before, we use the weighted averages to get the actual GI, finally giving us:

$$= \left(\frac{1}{1+6}\right) * 0 + \left(\frac{6}{1+6}\right) * 0.5 = \mathbf{0.429}$$

We now do the same for all the other average Age values. When we do this, we get the result in Table 5.05 below.



9.5	Age	0.429
15	7	0.343
26.5	12	0.476
36.5	18	0.476
44	35	0.343
66.5	38	0.429
	50	
	83	

Table 5.05. (Accessible table).

We can see from the arrow identification above that there are two possible candidates for the threshold values, namely 15 and 44, as they have the lowest GI value. Therefore, for our next branch we can choose either one, but if you have a singular GI value, you would use that as the sole branch. Here, however, we will choose 15. Running through the same process once more results in Figure 5.10.

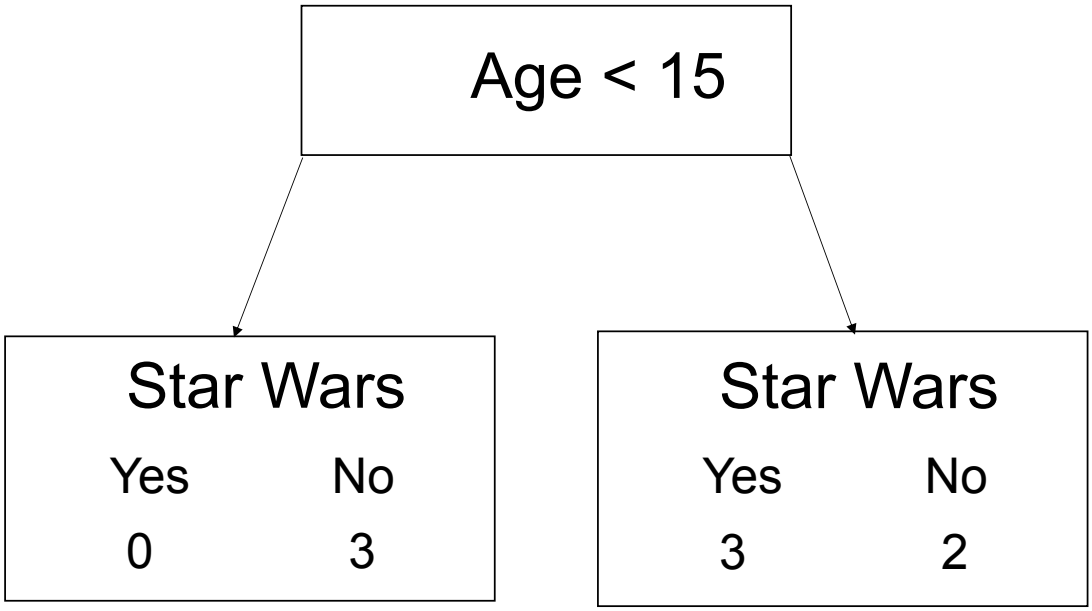


Table 5.10. (Accessible text) (Accessible list).

We must not forget, however, that we are trying to determine here which variable we should choose as the best variable to use as the root of our classification tree, by comparing the GI of Nachos, Sprite and Age.

By completing and comparing GIs, we see that Sprite has the lowest GI; therefore, this is the root of our tree that will give us the best classification tree for any future data points. The people that like Sprite go to the branch on the left and the people that do not like Sprite go to the branch on the right, as was shown in Figure 5.07, reproduced below.

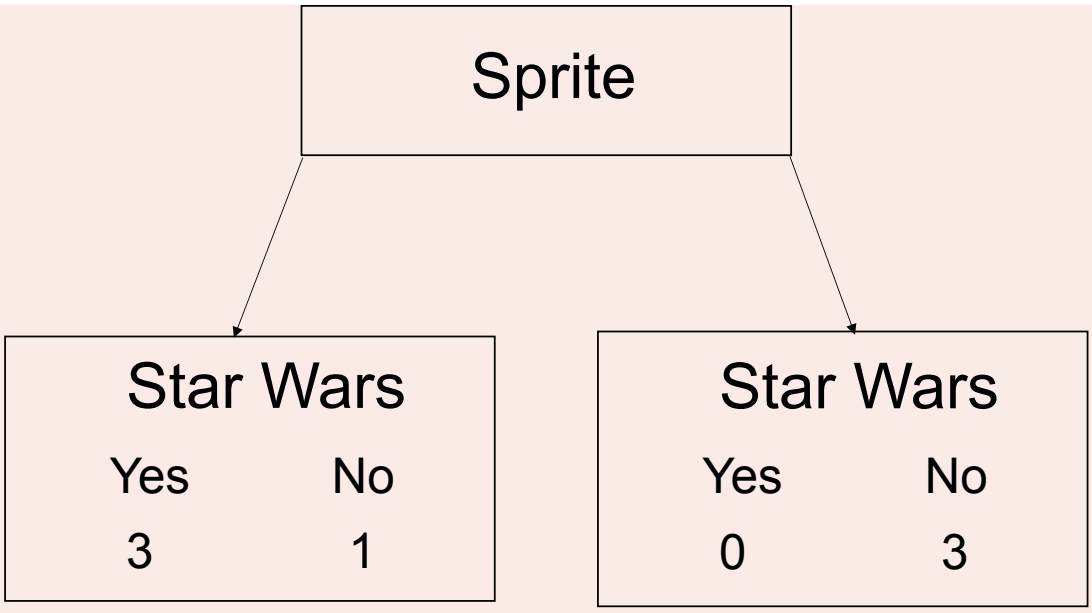


Figure 5.07. The completed classification tree using the variable Sprite by Arden University. (Accessible text). (Accessible list).

Notice from the above that all four people that like Sprite are within this leaf. Three people like Star Wars and one does not. As this leaf is ***impure***, let us determine whether we can reduce the impurity by adding the variables Nachos and Age to that leaf. If we follow the same process as we did previously to compare the GIs of these leaves, we find that the left leaf will have a GI of 0.25 and the GI for Age < 12.5 is 0, so we choose the leaf Age < 12.5. This is shown in Figure 5.11.

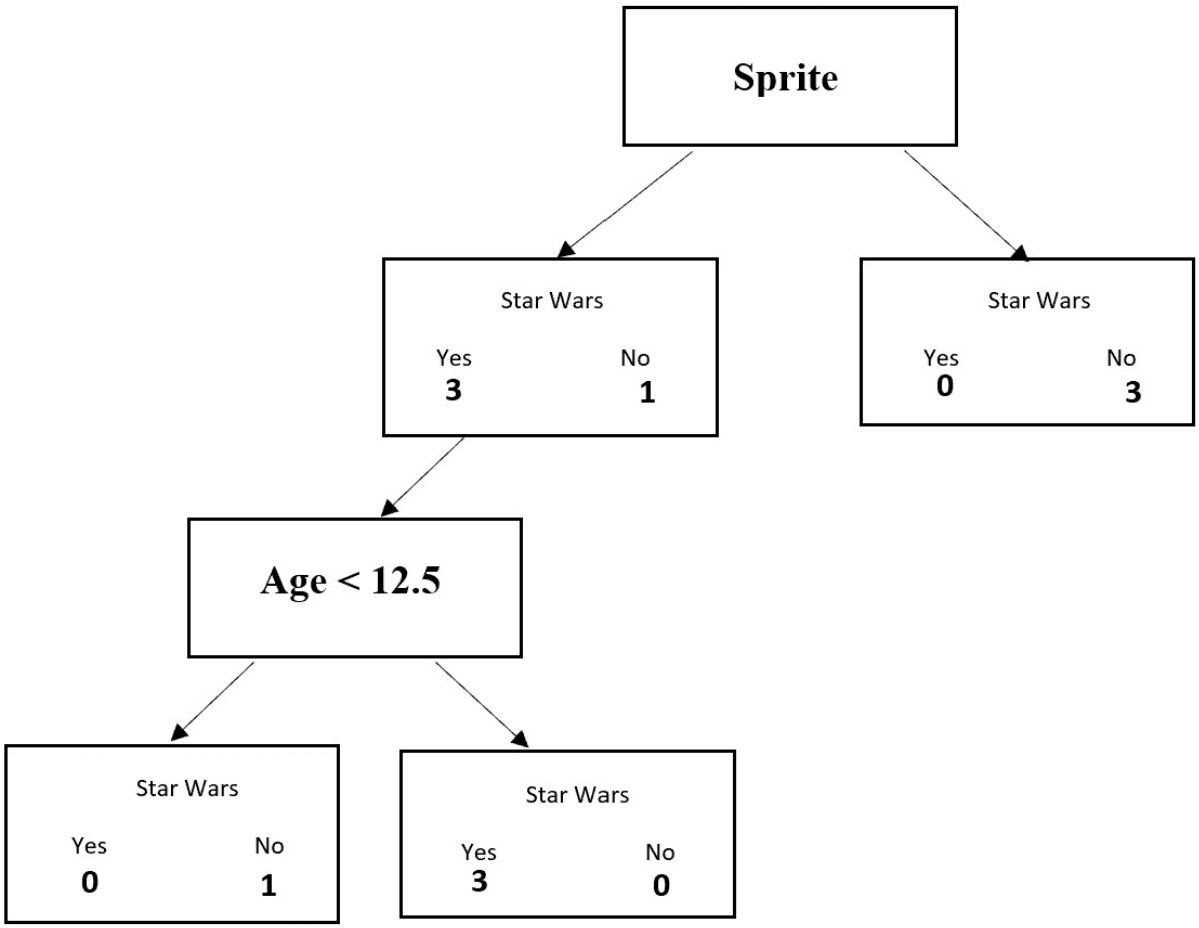


Figure 5.11. Combining both singular trees to begin building a larger singular tree by Arden University. (Accessible list).

The final thing we need to do before we can complete the construction of the finalised classification tree is assign output values for each leaf. As a rule of thumb, the output value for a leaf is whatever variable has the most values. Therefore, the completed tree can be categorised as shown in Figure 5.12.

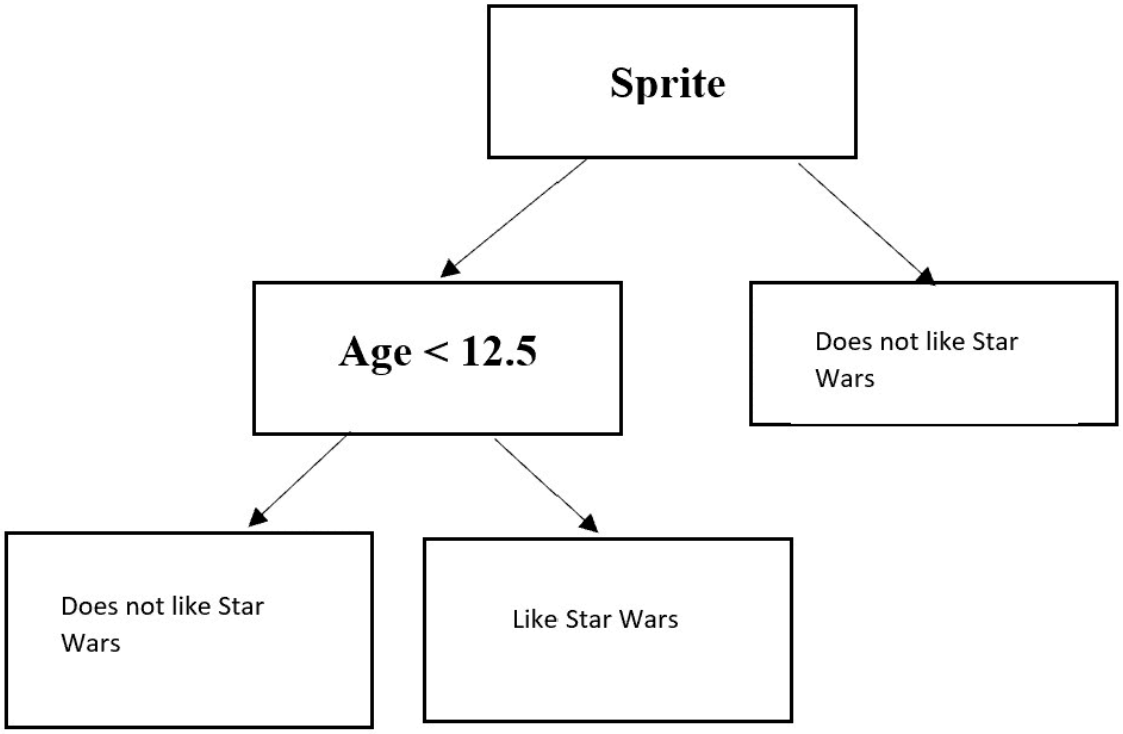


Figure 5.12. Combining both singular trees to begin building a larger singular tree by Arden University. (Accessible list).

The classification tree is now complete! If we were to categorise a new person in the dataset, we could run them through the tree to classify. To test this, let us add a new person/data point to the dataset that has the inclusive data shown in Table 5.06.

Likes Nachos	Likes Sprite	Age	Likes the film "Star Wars"
Yes	Yes	15	??

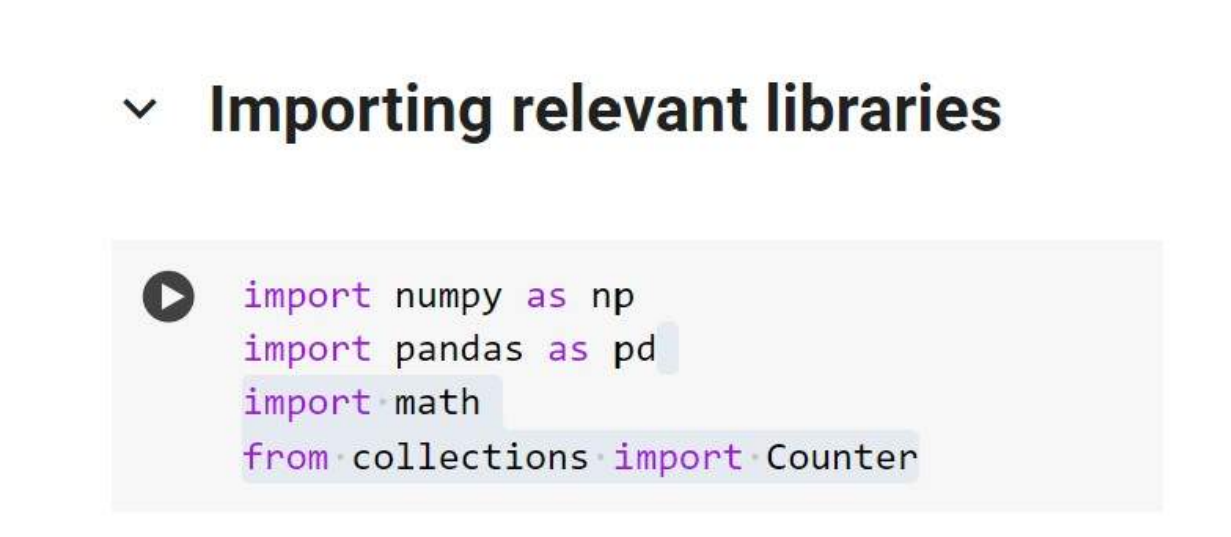
Table 5.06.

If we flow the new data point through the tree, we would determine that the new person would like Star Wars. Hurrah!

Now that we have run through the classification decision tree in a step-by-step manner, let us look at a code that demonstrates the implementation of a classification tree employing entropy and information gain. Please remember that the main reason we engage in this model is to find the optimum feature (leaf) in which to split, therefore generating the largest information gain result. The best way to do this is to use GI and Entropy, which we have just detailed above.

Continue

Example



Screenshot 1a from Google Colab by Arden University. (Accessible code).

```
[ ]  # In this example, to complete the study there would need to be a recursive call
     # to keep finding the best split for every new data subset.
     # However, that does get rather complex quite quickly and the goal of this example is to
     # practice the idea behind decsion trees in general, how to calculate the entropy and
     # Information Gain.
```

Screenshot 1b from Google Colab by Arden University. (Accessible code).

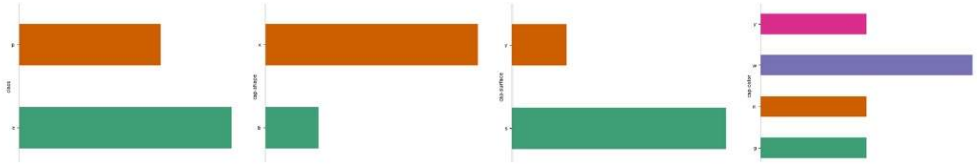
Checking the data set has been uploaded correctly

```
[2]  df = pd.read_csv("/content/Mushroom data set.csv")
     df.head()
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring
0	p	x	s	n	t	p	f	c	n	k	...	s	w
1	e	x	s	y	t	a	f	c	b	k	...	s	w
2	e	b	s	w	t	l	f	c	b	n	...	s	w
3	p	x	y	w	t	p	f	c	n	n	...	s	w
4	e	x	s	g	f	n	f	w	b	k	...	s	w

5 rows × 23 columns

Categorical distributions



Screenshot 1c from Google Colab by Arden University. (Accessible code).

Checking that there are no missing values

```
[6]  df.isnull().sum()
```

```
class                       0
cap-shape                   0
cap-surface                 0
cap-color                   0
bruises                     0
odor                        0
gill-attachment             0
gill-spacing                0
gill-size                   0
gill-color                  0
stalk-shape                 0
stalk-root                  0
stalk-surface-above-ring    0
stalk-surface-below-ring    0
stalk-color-above-ring      0
stalk-color-below-ring      0
veil-type                   0
veil-color                  0
ring-number                 0
ring-type                   0
spore-print-color           0
population                  0
habitat                     0
dtype: int64
```

Screenshot 1d from Google Colab by Arden University. (Accessible code).

Checking how many features there are in the data set

```
[8]  print('{} Data features'.format(len(df.columns)))

     23 Data features
```

Screenshot 1e from Google Colab by Arden University. (Accessible code).

Setting up the Entropy and Information Gain

```
[9]  def entropy(labels):
         entropy=0
         label_counts = Counter(labels)
         for label in label_counts:
             prob_of_label = label_counts[label] / len(labels)
             entropy -= prob_of_label * math.log2(prob_of_label)
         return entropy

     def information_gain(starting_labels, split_labels):
         info_gain = entropy(starting_labels)
         for branched_subset in split_labels:
             info_gain -= len(branched_subset) * entropy(branched_subset) / len(starting_labels)
         return info_gain
```

Screenshot 1f from Google Colab by Arden University. (Accessible code).

Generating the Decision Tree for Discrete Variables

```
def split(dataset, column):
    split_data = []
    col_vals = df[column].unique()
    for col_val in col_vals:
        split_data.append(dataset[dataset[column] == col_val])
    return(split_data)
```

Screenshot 1g from Google Colab by Arden University. (Accessible code).

Decision Tree optimal split based on Entropy and Information Gain

```
def find_best_split(dataset):
    best_gain = 0
    best_feature = 0
    features = list(dataset.columns)
    features.remove('class')
    for feature in features:
        split_data = split(dataset, feature)
        split_labels = [dataframe['class'] for dataframe in split_data]
        gain = information_gain(dataset['class'], split_labels)
        if gain > best_gain:
            best_gain, best_feature = gain, feature
    print(best_feature, best_gain)
    return best_feature, best_gain

new_data = split(df, find_best_split(df)[0]) # contains a list of dataframes after splitting
```

```
odor 0.9060749773839998
```

Screenshot 1h from Google Colab by Arden University. (Accessible code).

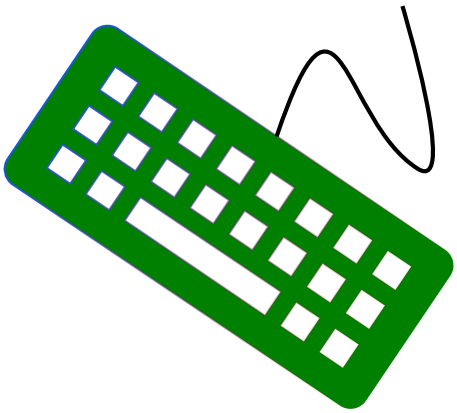
Hurrah!

Continue

Topic 1 - Activity 1

Titanic data set

Using the “titanic” data set provided, create a decision tree model with the targeted variable “survived”. Import the data correctly into your IDE and take the required steps to pre-process it before creating the classification model. Once this has been achieved, construct a decision tree machine-learning model and assess the accuracy of the model. If the accuracy is below 90%, take the appropriate step to improve the model’s accuracy.



Continue to solution

DOC

Topic 1 Activity solution.docx

336.8 KB

Continue

Topic 1 - Activity 2

Reading

- Chapter 1: The IDLE Integrated Development Learning Environment (pp. 23-40) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 3: Unsupervised Learning and Preprocessing (pp. 103-151) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.
- Chapter 2: Python Language Basics, IPython, and Jupyter Notebooks (pp, 30-56) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney.
- Chapter 1: Data, Stats and Stories – An Introduction (pp. 39-71) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue to Topic 1 summary

TOPIC REVIEW

Within the first topic, we have focused on decision trees. In addition, we have also demonstrated, both theoretically and numerically, information gain. We initially gave a detailed demonstration of how a decision tree algorithm works and the improved efficiency process that is involved in producing, as far as possible, the optimum model. Once the numerical example had been given, a coded example was detailed to enable a real-world example using a famous data set.

Hopefully, you now have a much better understanding of how a decision tree works and, more importantly, why we would adopt one within a supervised machine-learning model. Now that Topic 1 has been completed, we can move further into the process of decision trees by providing an additional technique that can add even more efficiency within a decision tree algorithm: gain ratio.

Continue to Topic 2

Topic 2: Gain ratio

Image from Articulate Rise 360 Content Library.

Did you know?

In 2023, the UK government announced it would invest £500 million into AI and machine learning computing capacity over the next two years (Deutscher, 2023).

Image from Articulate Rise 360 Content Library.

Image from Articulate Rise 360 Content Library.

Over 432,000 companies use AI and machine learning in the UK (Department for Digital, Culture, Media & Sport, Office for Artificial Intelligence and Department for Science, Innovation & Technology, 2022).

Continue

As we discovered in the previous topic, information gain is primarily used within classification algorithms, especially decision trees. The process splits attributes within the decision tree to find the most appropriate and efficient split. In this topic, however, we will demonstrate, using both theory and example, the alternative to information gain, one which aims to remove, wherever possible, the bias that can be evident in an information gain investigation: gain ratio.

Say, for example, we have two identified attributes (colour and size) from a “Fruit” dataset. The aim is to build a decision tree that will predict new fruit that we introduce, based on the identified two attributes. The colour attribute has three possible outcomes: red, yellow, and orange. The size attribute has two outcomes: small and large.

If we were to theoretically run this through a decision tree algorithm and the highest information gain result determined that colour would be the best attribute to split on, then that is what we would do. However, this might not be entirely accurate, as size could be the best attribute, due to it having fewer outcomes. This is where we would turn to gain ratio, as it takes into account both attributes and their associated outcomes, and it measures both the information gain and the number of outcomes of a feature to determine the best split. Gain ratio achieves this by normalising the information gain by the split information, using the following equation:

$$-\sum_{i=1}^{n} D_i log_2 D_i$$

where D_i is the probability for every *I* outcome from within the attribute.

Let's put this into a fictitious example. We want to know whether a cricket match should be played based on the following attributes: temperature, humidity, and wind. Temperature has three features (outcomes): hot, mild, and cool. Humidity has the following two features: high and normal. Finally, wind has two features: weak and strong. The target feature that we want to determine is play cricket, either yes or no. An example of the dataset is shown below.

Temp	Humid	Wind	Play Cricket
Hot	High	Weak	N
Hot	High	Strong	N
Mild	High	Weak	Y
Cool	Normal	Weak	Y
Cool	Normal	Strong	N

Table 5.07.

To determine the split information for the temperature attribute, we sum all the instances; in this example, there are five. Next, we determine the number of instances within temperature; this will give hot = 2, mild = 1 and cool = 2. We can now insert this information into the given formula.

$$-\sum_{i=1}^{n} D_i log_2 D_i$$

$$-\frac{2}{5} * log_2 * \frac{2}{5} - \frac{1}{5} * log_2 * \frac{1}{5} - \frac{2}{5} * log_2 * \frac{2}{5} = 0.92$$

We can also calculate the split information on the other attributes, namely, humidity and wind. Once these have been determined, we can then put the results into the gain ratio formula, which is:

$$Gain\ ration = \frac{information\ gain}{split\ info}$$

We have not completed the final steps here, as we are going to demonstrate a complete example, both numerically and in Python. The above is to demonstrate and give some idea of the steps that are involved when engaging with gain ratio. The interpretation of the result is that the highest gain ratio is the optimum feature on which to split.

Continue

Example

In the following example, we will be using a fictitious fruit dataset and we want you to use gain ratio to increase the accuracy in determining whether the fruit is bitter or not. The fictitious dataset that we will be using in this example is shown below.

Fruit	Colour	Shape	Weight	Bitter
1	red	round	heavy	N
2	red	round	light	Y
3	yellow	round	heavy	Y
4	yellow	round	light	Y
5	yellow	round	light	N
6	yellow	square	heavy	N
7	orange	round	heavy	N
8	orange	square	heavy	N
9	orange	square	light	N
10	orange	square	light	N

Table 5.08.

As previously mentioned, the goal is to build a decision tree based on the best split using gain ratio. To achieve this, we apply the following steps.

Step 1

We have to initially determine the entropy of the targeted (parent) variable “bitter”, so that we can then calculate the gain ratio of all other features to determine the “bitter” variable. To do this, we use the following probability formula, where p represents probability:

$$Entropy(Parent) = -p(yes)log_2p(yes) - p(no)log_2p(no)$$

The information from the dataset gives us $p(\text{yes}) = 3 / 10 = 0.3$ are bitter, $p(\text{no}) = 7 / 10 = 0.7$ are not bitter. Putting this into our formula results in:

$$Entropy(Parent) = -(0.3)log_2(0.3) - (0.7)log_2(0.7) = 0.881$$

Step 2

Having now calculated the entropy of the parent variable “bitter”, we can determine the information gain (IG) by comparing this against the entropy weighted averages of the child nodes using the following formula:

$$IG = Entropy(Parent) - entropy\ weighted\ averages\ (child)$$

To get the weighted averages of the child nodes, we sum up the product of the probabilities and its associated entropy in the following manner:

$$weighted\ average\ of\ entropy\ (child) = sum\ (p(child) * entropy(child))$$

We how have everything we need to be able to calculate the IG for each feature (i.e., colour, shape and weight). If we begin with colour, we have three values: red, yellow, and orange. To calculate the entropy of the first child node (i.e., red), we first need to determine the probability of the targeted variable “bitter” for the red child node only, by counting the number of fruits that are red, plus bitter, then dividing by the total number of red fruits. So that we can make a clear distinction, we will call

this probability p(yes / red). Conversely, we also need to calculate the probability of a red fruit that is not bitter, and we will call this probability *p*(*no* / *red*). Therefore:

$$p\left(\frac{yes}{red}\right) = \frac{1}{2} = 0.5$$

$$p\left(\frac{no}{red}\right) = \frac{1}{2} = 0.5$$

We can now use the above probabilities to determine the entropy of the red (child) node using the same formula we have consistently used, as shown below:

$$entropy(red) = -p\left(\frac{yes}{red}\right) * log_2\, p\left(\frac{yes}{red}\right) - p\left(\frac{no}{red}\right) * log_2 p\left(\frac{no}{red}\right)$$

$$= -(0.5) * log_2(0.5) - (0.5) * log_2(0.5) = 1$$

Now calculate the entropy of the other child nodes in the same manner, by counting the number of fruits associated with a child node that are bitter or not, then dividing by the total number of fruits belonging to that child node, as shown below:

entropy(*yellow*) = −(0.5) * log_2(0.5) − (0.5) * log_2(0.5) = 1

entropy(orange) = −(0.25) * log_2(0.25) − (0.75) * log_2(0.75) = 0.881

p(*red*) = 0.2

p(*yellow*) = 0.4

p(*orange*) = 0.4

We now have everything we need to be able to calculate the average weighted child entropies, as shown below:

$$entropies(children) = (0.2) * (1) + (0.4) * (1) + (0.4) * (0.881) = 0.922$$

$$IG(colour) = 0.881 - 0.922 = -0.041$$

As with the above, we can determine the IG for shape and weight in the same manner.

Step 3

For each feature in the dataset, we need to calculate the split information. To do this, we add the negative probabilities of the child nodes, then multiply by those associated probabilities:

$$split\ info(colour) = -sum(p(child) * log_2(p(child))$$

$$= -(0.2) * log_2(0.2) - (0.4) * log_2(0.4) - (0.4) * log_2(0.4) = 1.371$$

We continue this process in the exact same manner to determine the split information for both shape and weight.

Step 4

In the final step, we need to divide the IG by the split information for all features, in order to determine the gain ratio:

$$Gain\ ration(colour) = \frac{IG(colour)}{split\ Info(colour)} = -\frac{0.41}{1.371} = -0.03$$

In the same way, we can determine the gain ratio for shape and weight. Once we have completed all the required calculations, we can compare the gain ratios for each feature. The feature that has the highest gain ratio is recognised as the best feature for splitting, therefore optimising, as much as possible, the decision tree algorithm.

Continue

Coded example

We will now demonstrate an example of using the gain ratio to find the optimum feature using the famous iris dataset, which has been directly imported from the sklearn repository.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
```

Screenshot 1a from Google Colab by Arden University. (Accessible code).

```
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

Screenshot 1b from Google Colab by Arden University. (Accessible code).

```
# Create the decision tree classifier
clf = DecisionTreeClassifier(random_state=0, criterion='gini')
clf.fit(X, y)
```

```
DecisionTreeClassifier(random_state=0)
```

```
[10] # Get the feature importances (which include the gain ratio)
     importances = clf.feature_importances_
```

Screenshot 1c from Google Colab by Arden University. (Accessible code).

```
[10] # Get the feature importances (which include the gain ratio)
     importances = clf.feature_importances_
```

```
gain_ratios = {}
# Print the gain ratio for each feature
for i, feature in enumerate(iris.feature_names):
    gain = importances[i]
    split = clf.tree_.impurity[i]
    gain_ratio = gain / (split + 1e-4)
    print(f'GI ratio for {feature}: {gain_ratio:.2f}')

    gain_ratios[feature] = gain_ratio
```

```
GI ratio for sepal length (cm): 0.00
GI ratio for sepal width (cm): 133.33
GI ratio for petal length (cm): 0.13
GI ratio for petal width (cm): 5.49
```

Screenshot 1d from Google Colab by Arden University. (Accessible code).

```
# Find the feature with the highest gain ratio
best_feature = max(gain_ratios, key=gain_ratios.get)

# Print the best feature for splitting
print(f'Best feature for splitting: {best_feature}')
```

```
Best feature for splitting: sepal width (cm)
```

Screenshot 1e from Google Colab by Arden University. (Accessible code)

Hurrah!

Continue

Topic 2 - Activity 1

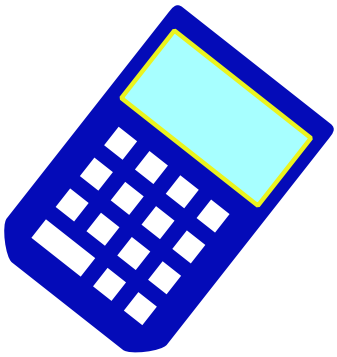
Calculation

Using the following gain ratio formula

$$-\sum_{i=1}^{n} D_i log_2 D_i$$

calculate the gain ratio in Python of an attribute that has seven features and 23 entries, plus 13 features and 23 entries.

Calculate the gain ratio in Python of an attribute that has 24 features and 100 entries, plus 35 features and 100 entries.



Continue

Topic 2 - Activity 2

Reading

- Chapter 1: The IDLE Integrated Development Learning Environment (pp. 23-40) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 3: Unsupervised Learning and Preprocessing (pp. 103-151) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.
- Chapter 2: Python Language Basics, IPython, and Jupyter Notebooks (pp, 30-56) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney.
- Chapter 2: Python Programming Primer (pp. 71-136) from Statistics and Data Visualisation with Python by Rogel-Salazar

Continue

TOPIC 2 SUMMARY

In Topic 2, we looked at an improved machine-learning technique for the information gain process within a decision tree: gain ratio. Hopefully, you can determine that with all complex processes within a machine-learning environment, trade-offs can be expected, and gain ratio is no exception. Therefore, a balance must be made between overfitting and underfitting the model. Gain ratio, however, does limit, wherever possible, the biases that can be incurred when applying an information gain process to a classification model. It is, therefore, a worthwhile process to undertake.

Decision trees and their associated techniques to improve efficiency and accuracy are now almost complete within this module. However, before we move onto the

final accuracy techniques, it would be worth considering the random forest algorithm, which is the following topic.

Continue to Topic 3

Topic 3: Random forest

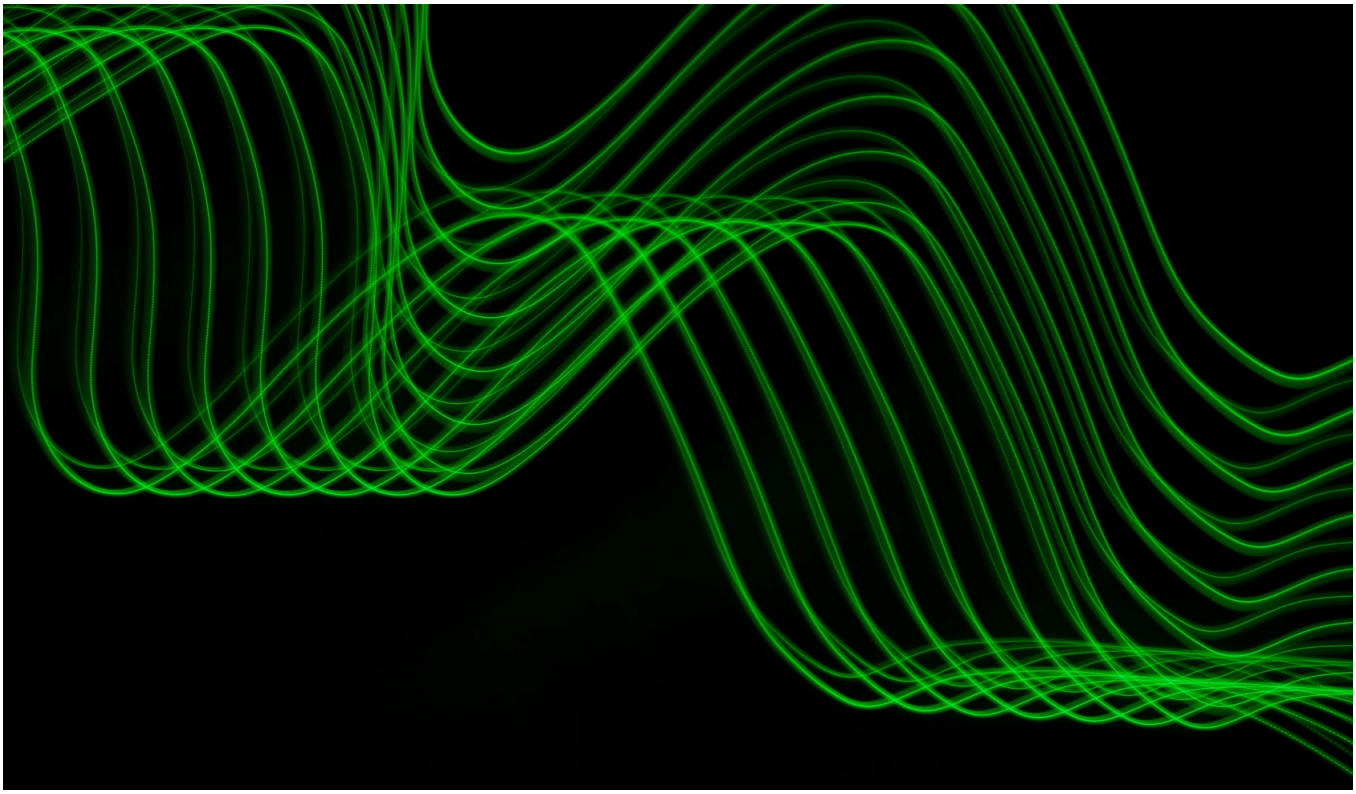


Image from Articulate Rise 360 Content Library.

Did you know?

With the increased use of big data, automation, AI and machine learning, jobs for data analysts are estimated to grow by 23% between 2021 and 2031 (London School of Economics and Political Science, 2023).

Image from Articulate Rise 360 Content Library.

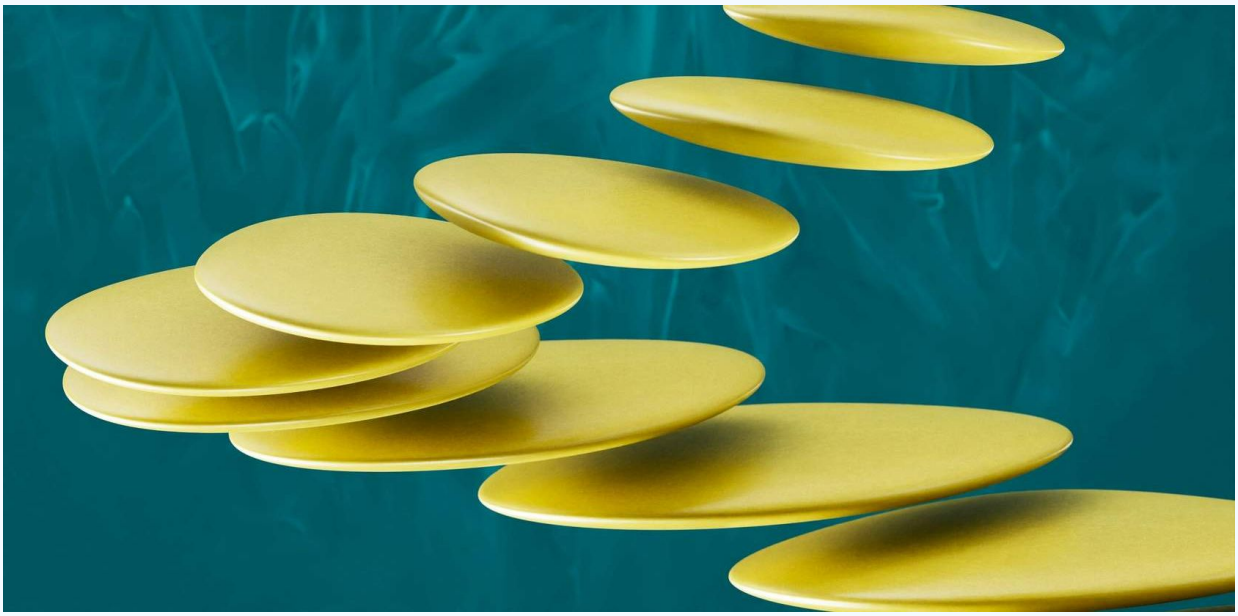


Image from Articulate Rise 360 Content Library.

Machine-learning technologies have the capability to produce more than £20 trillion annually in revenue by 2025 (Mrkonjić, 2022).

Continue

As we demonstrated in the previous topic, the decision tree is a classification method that enables the analyst to easily classify new data, once the algorithm has been configured correctly. As a reminder, see Figure 5.13.

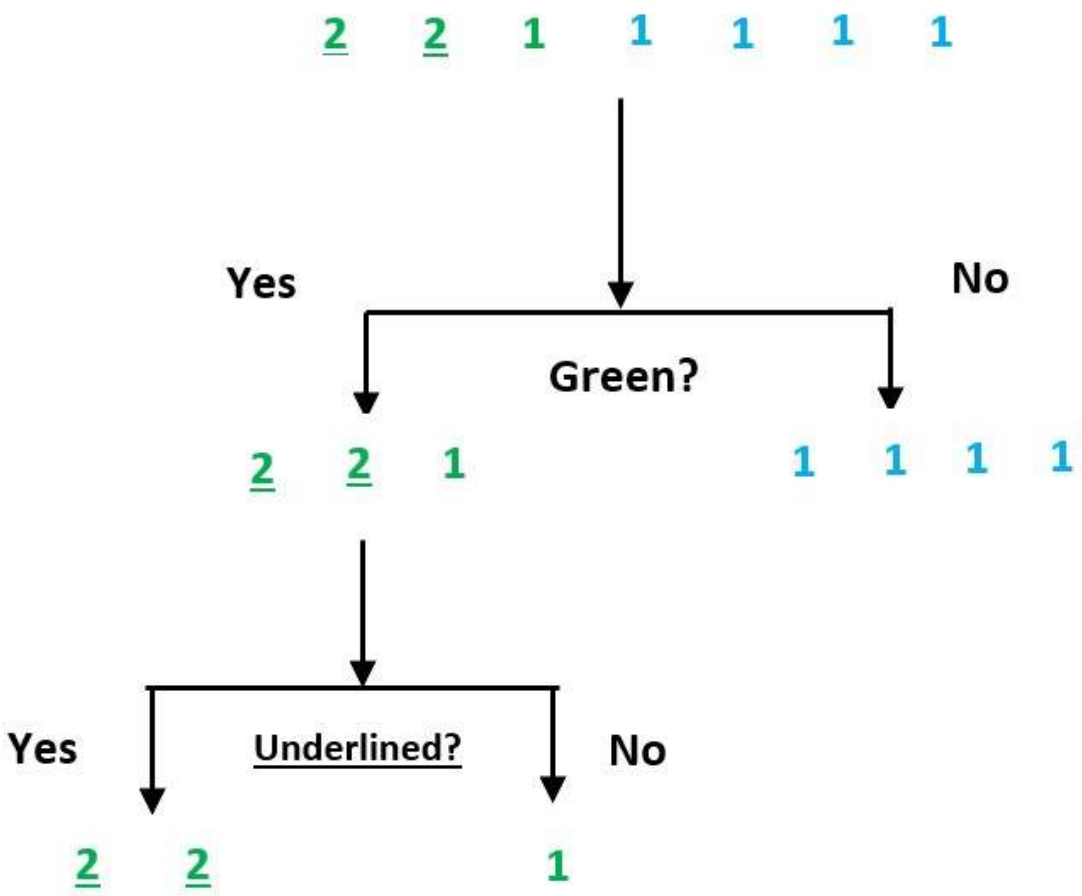


Figure 5.13. A simple classification tree by Arden University. (Accessible list).

Random forest, however, consists (as the name implies) of multiple classification trees operating in unison, thereby mitigating errors that can occur, thus giving a more accurate result. Each classification tree in the random forest algorithm gives a class prediction, and the class with the most votes becomes our model's prediction. This is shown in Figure 5.14:

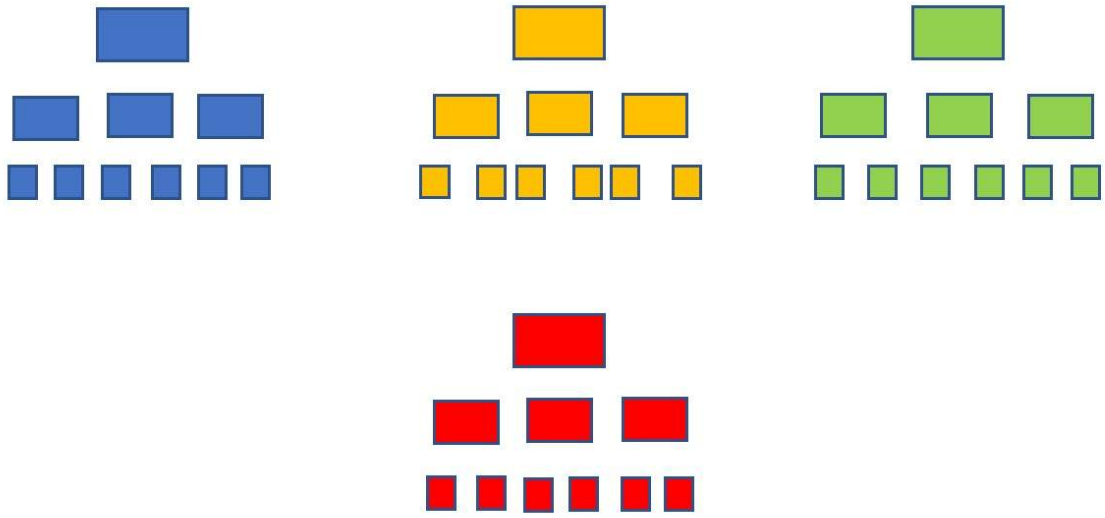


Figure 5.14. Random forest classification schema by Arden University.

The concept behind random forest is simple, yet powerful: many minds are greater than one. In essence, the random forest model works well because **a large number of relatively uncorrelated trees operate as a unified entity, and the entire entity will outperform any individual tree. As such, the trees protect each other from individual errors:** some trees may be wrong, many other trees will be right, so, as a forest, the trees are able to move in the correct direction.

Continue

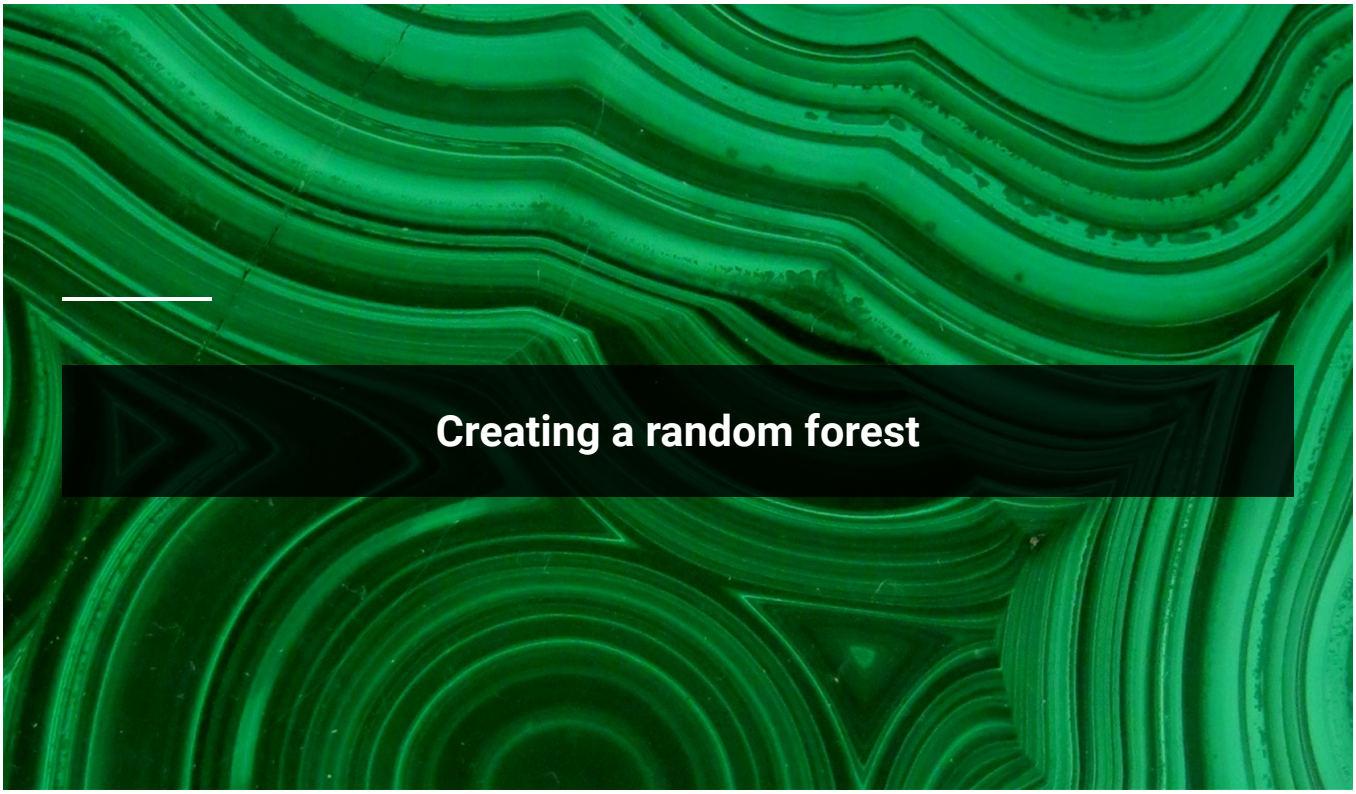


Image from Articulate Rise 360 Content Library.

To demonstrate the construction of a random forest, we will use a completely fictitious dataset on heart disease, shown below:

Chest tightness	Good circulation	Blocked valves	Weight (lbs)	Heart failure
No	No	No	132	No
Yes	Yes	Yes	201	Yes

Chest tightness	Good circulation	Blocked valves	Weight (lbs)	Heart failure
Yes	Yes	No	178	No
Yes	No	Yes	223	Yes

Table 5.10.

From the data, the first step is to bootstrap the dataset, ensuring that the bootstrapped dataset is the same size as the original. To achieve this, we randomly select samples from the original dataset. The important detail when constructing a bootstrapped dataset is that we are allowed to choose the same sample more than once. The first sample we randomly select from the original dataset will be row two. The second randomly selected sample will be from row one, the third will be from row four, and the fourth sample will again be from row four. We have now constructed the bootstrapped dataset from the original dataset. This is shown below:

Chest tightness	Good circulation	Blocked valves	Weight (lbs)	Heart failure
Yes	Yes	Yes	201	Yes
No	No	No	132	No
Yes	No	Yes	223	Yes
Yes	No	Yes	223	Yes

Table 5.10.

The second step is to create a classification tree from the data in the bootstrapped dataset. However, we only use a random subset of variables at each step. For the purpose of this example, we will only be considering two randomly chosen variables at each step; initially, these will be circulation and blocked valves. It could, however, have been any combination of the four variables.

To determine which variable out of circulation and blocked values did the best job in separating the samples, we would have to go through the exact same process as we did when we constructed the classification tree in the previous topic. But, as we have already been through this process, in this example we will randomly choose only the circulation variable as our root. We can now concentrate on the remaining variables as we make our way through the first classification tree, randomly selecting the remaining variables as branches, just as we did in the classification tree topic. When we have worked our way through, the resulting classification tree may look like Figure 5.15.

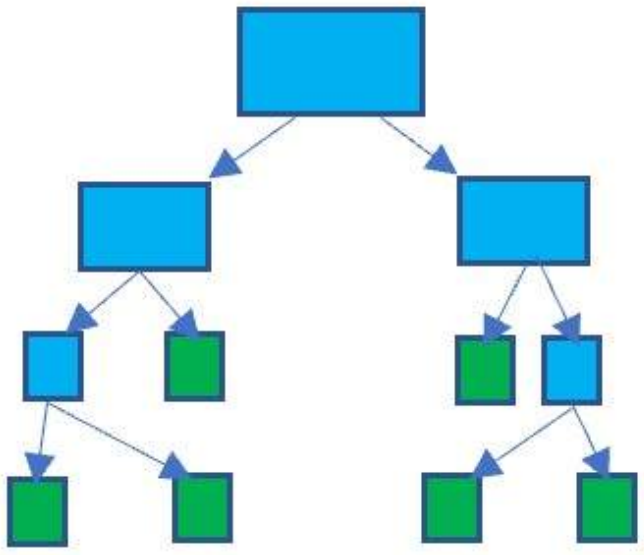


Figure 5.15. The completed initial classification tree using the circulation variable by Arden University. (Accessible list).

So, we have now constructed a classification tree from a bootstrapped dataset and only considered a random subset of variables at each step. To build the forest, we return to the first step by making a new bootstrapped dataset and building the corresponding second tree in the manner we have just described, considering a subset of variables at each step. By repeating the steps a further five times, the forest will have been constructed, and it will be full of different classification trees, as shown in Figure 5.16.

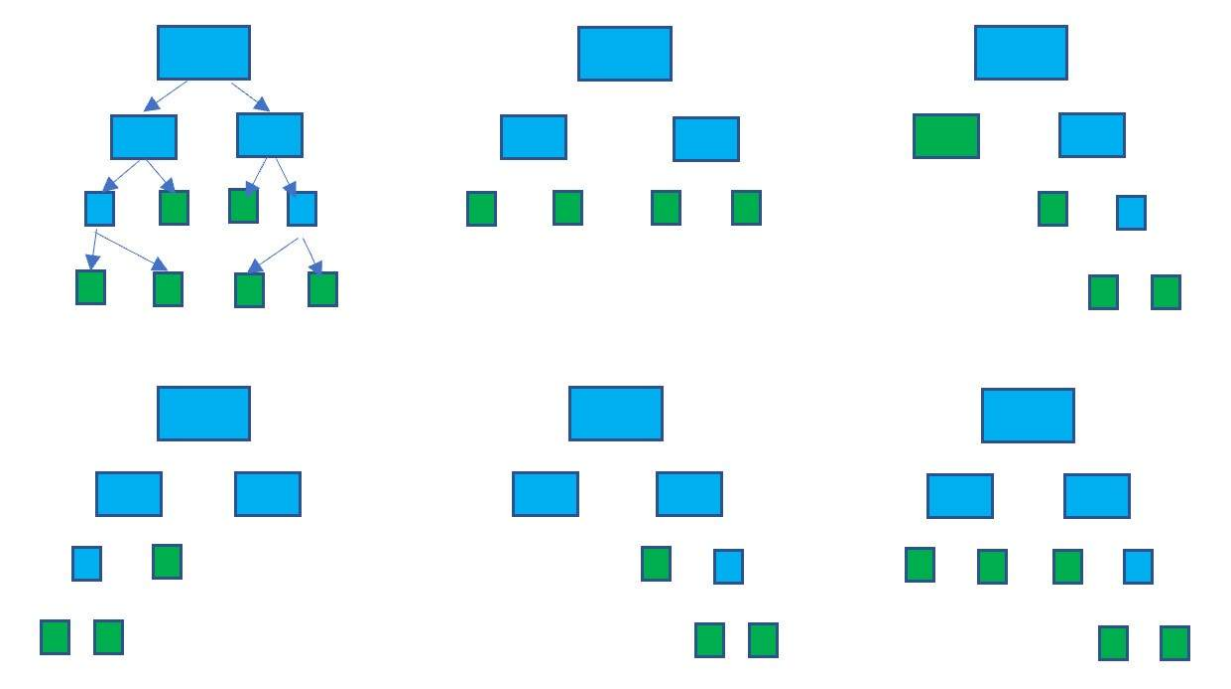


Figure 5.16. A possible result of individual classification tree making a random forest by Arden University.

By using multiple bootstrapped sample datasets and considering only subsets of the different variables at each step, we have constructed a wide variety of trees. It is this variety which makes the random forest far more effective in giving the correct result than a single classification tree. The analysis, or how we use the random forest, comes into play when we add a new data point that we want to classify. So, if we use the new data point below, we can send it through the forest to determine whether the person has heart disease.

Chest tightness	Good circulation	Blocked valves	Weight (lbs)	Hear failure
Yes	No	No	175	

Table 5.11.

The new data point is run through the first tree in the forest, and this tree may classify the new data as ***Yes***. The data is now run through the second tree, and this may result in a classification of ***Yes*** also. The third, fifth and sixth all result in ***Yes***. The only tree to result in ***No*** is the fourth tree. As ***Yes*** received more votes, we can confidently classify the new data as ***Yes***: the patient will have heart failure.

This is all well and good, but how can we justify our results as being accurate or not? To do this, we have to go back to our bootstrapped datasets. If you remember, for each bootstrapped dataset we constructed, there was always one data point that was missing from the original data. Therefore, we run the omitted data point through the entire forest again and take the highest vote number as our classification result. Finally, we run all the remaining omitted data points through the forest and take the average result. Ultimately, we can measure how accurate our random forest is by the proportion of omitted data samples that were correctly classified by the entire random forest. Hurrah!

We now know how to create a random forest in Python and how to use a random forest algorithm to evaluate accuracy, using the commonly used Scikit library and the famous iris dataset to classify flower types.

Continue

Example

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
iris = load_iris()
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
%matplotlib inline
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

Screenshot 1a from Google Colab by Arden University. (Accessible code).

```
print(iris.target_names)
```

```
['setosa' 'versicolor' 'virginica']
```

```
print(iris.feature_names)
```

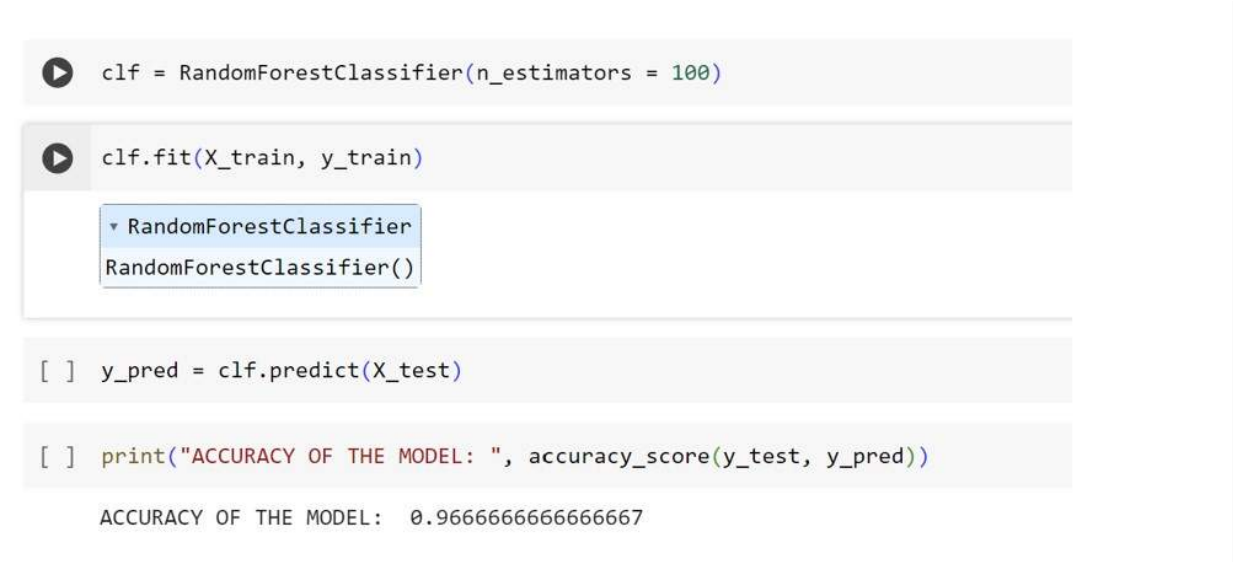
```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

```
X, y = load_iris( return_X_y = True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.40)
```

```
data = pd.DataFrame({"sepal length (cm)": iris.data[:, 0], "sepal width (cm)": iris.data[:, 1],
            "petal length (cm)": iris.data[:, 2], "petal width (cm)": iris.data[:, 3],
            "species": iris.target})
```

Screenshot 1b from Google Colab by Arden University. (Accessible code).



Screenshot 1c from Google Colab by Arden University. (Accessible code).

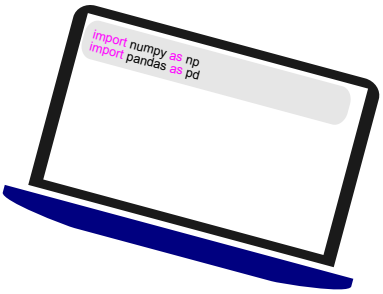
The result means that we have created a random forest model in Python and run the omitted data points through the forest, and the code has produced a random forest classifier with accuracy of 96%, which is pretty good for a first attempt on a 60/40 training and test split.

Hurrah!

Continue

Topic 3 - Activity 1

Import the weather dataset, which is in the topic resources, and construct a random forest classifier. You will need to use on-hot encoding to change some categorical data to numerical data. When you have accomplished this, print out your classifiers result to test for accuracy.



Continue

Topic 3 - Activity 2

Reading

- Chapter 2: Anaconda, Spyder and the Libraries NumPy, Matplotlib and SymPy (pp. 41-52) from Python for Scientific Computing and Artificial Intelligence by Lynch
- Chapter 3: Unsupervised Learning and Preprocessing (pp. 103-151) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.

- Chapter 3: Built-In Data Structures, Functions, and Files (pp. 57-88) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney
- Chapter 2: Python Programming Primer (pp. 71-136) from Statistics and Data Visualisation with Python by Rogel-Salazar

Continue

TOPIC 3 SUMMARY

Within the third topic, we have taken the humble classification tree and turned it into a forest of trees. Random forest trees are widely used within machine learning for obvious reasons. Hopefully, you can now determine why a collection of decision trees is much better than a singular classification decision tree. In essence, all singular trees protect each other from the wrong classification. As was stated at the beginning of the topic, random forest is a simple analysis tool, yet one that is very powerful.

We will now move onto our final topic in improving the accuracy and efficiency of a decision tree algorithm: pruning.

Continue to Topic 4

Topic 4: Pruning

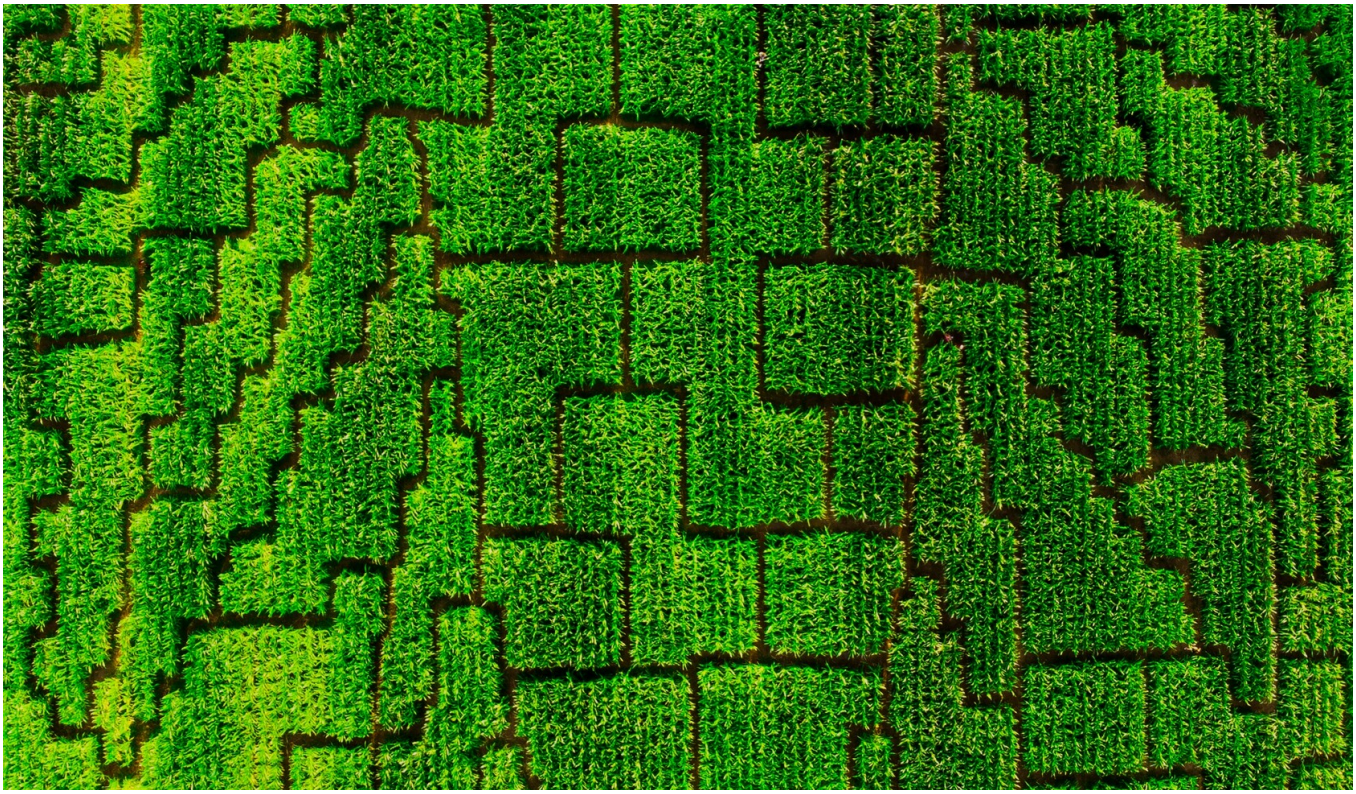


Image from Articulate Rise 360 Content Library.

Did you know?

The installation of AI and machine-learning systems into cars will increase by 109% in 2025 (Parkd, 2018).

Image from Articulate Rise 360 Content Library.

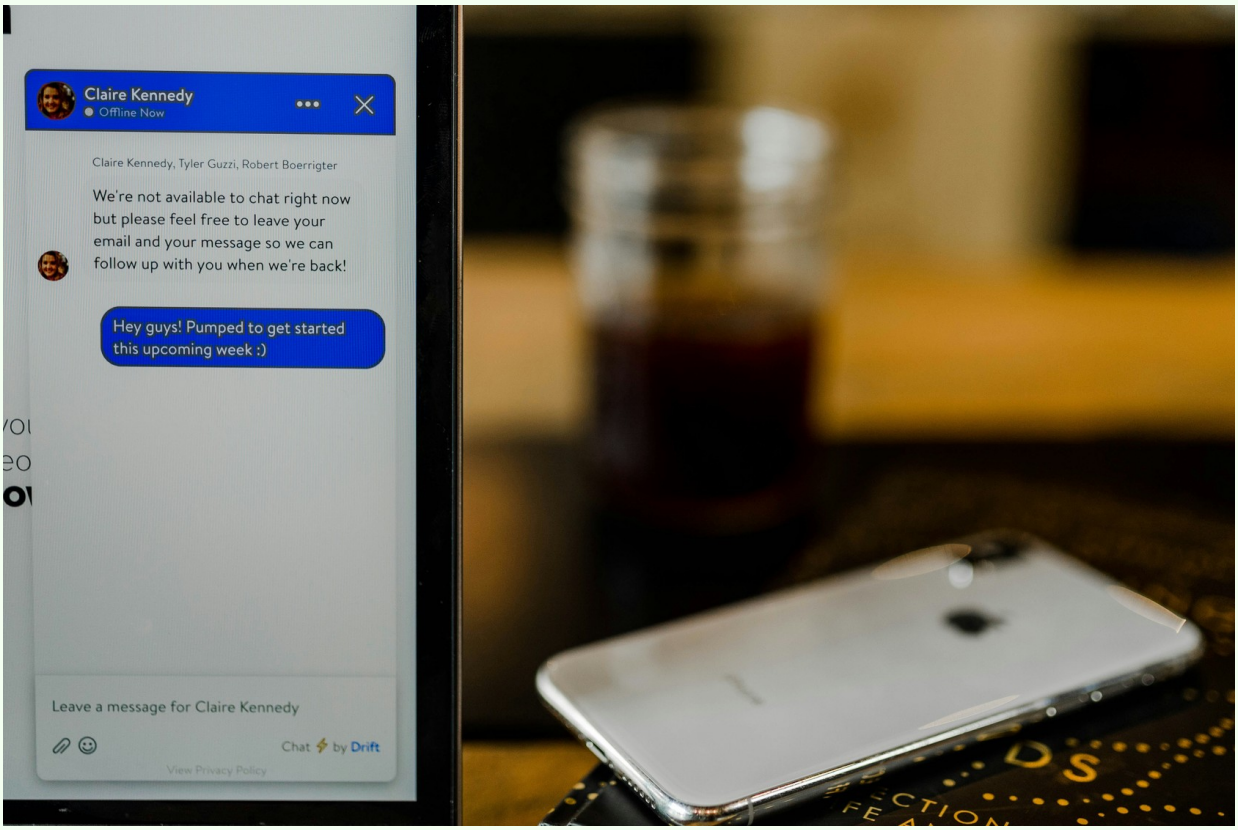


Image from Articulate Rise 360 Content Library.

In 2022, 73% of customer services used by organisations were conducted through AI and machine-learning applications (Wang, 2022).

Continue

Pruning is a machine-learning technique used to optimise the efficiency of a decision tree – in other words, it limits overfitting within the training data. Moreover, pruning aids in reducing the size of the decision tree by removing elements that are not really relevant to the model.

Of course, as with most machine-learning algorithms, there is a trade-off between overfitting and underfitting the data, so a balance must be sought between complexity and simplicity. As we may already know, the decision tree algorithm repeatedly partitions subsets into smaller subsets, until the final subsets are homogeneous in terms of the targeted outcome variable. This, in turn, means that, as far as possible, the model has learned the data exactly, or we would like to think so: this is the main reason why there are only a few data points in the final subsets. The caveat to that, however, is that if a new data point is introduced, it may not be categorised correctly.

There are three different forms of pruning, as briefly explained below:

1. **None:** No pruning is required. In other words, the model is performing as efficiently as possible, even if new data points are introduced.
2. **Minimum error:** With this process, the cross-validation error is at a minimum. In other words, the model is built with most of the data and then validated with the

remaining data. Therefore, as so far as test/train data it could possibly be an 80/20 split between test and training data.

3. **Smallest tree:** Here, the decision tree is pruned back slightly further than the minimum error. This then produces a smaller decision tree, and thus one that is slightly more tangible to an increase in any errors.

In summary, for the best accuracy, minimum error pruning without early stopping is considered the best initial approach to take. For a justified compromise between accuracy and an easy-to-read tree, it is suggested to apply small tree pruning without early stopping. Below, we have a coded example of pruning using the famous iris dataset once again, so that you can fully understand the processes involved within the creation and operation.

Continue

Pruning coded example

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import uniform, poisson
```

Screenshot 1a from Google Colab by Arden University. (Accessible code).

```
[2]  # load in the data
     data = load_iris()
```

```
[3]  X             = data.data
     y             = data.target
     class_names   = data.target_names
     feature_names = data.feature_names
```

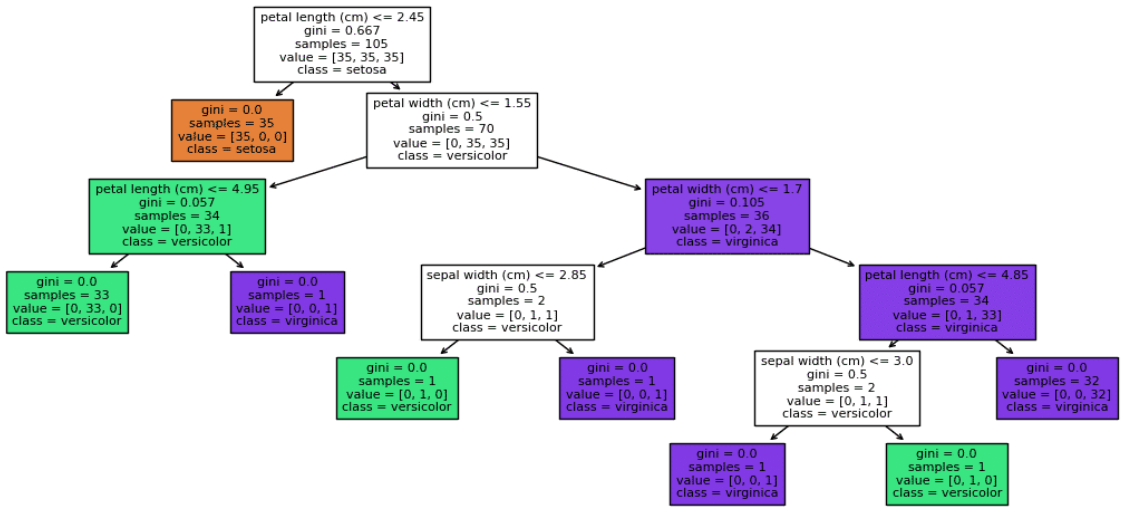
```
[4]  # perform a train-test split
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

```
     # fit a decision tree classifier
     clf = DecisionTreeClassifier(random_state=42)
     clf.fit(X_train,y_train)
```

```
▾        DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
[6]  # visualise the decision tree
     fig = plt.figure(figsize=(14,6))
     _ = plot_tree(clf,
                   feature_names=feature_names,
                   filled=True,
                   class_names=class_names,
                   fontsize=8)
```

Screenshot 1b from Google Colab by Arden University. (Accessible code).

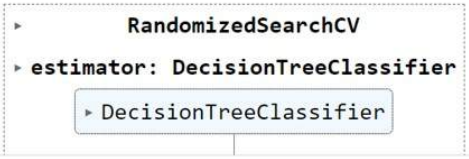


Screenshot 1c from Google Colab by Arden University.

```
# produce classification report
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        15
           1       1.00      0.80      0.89        15
           2       0.83      1.00      0.91        15

    accuracy                           0.93        45
   macro avg       0.94      0.93      0.93        45
weighted avg       0.94      0.93      0.93        45
```

```
[8] # setup parameter space
    parameters = {'max_depth':poisson(mu=2,loc=2),
                  'max_leaf_nodes':poisson(mu=5,loc=5),
                  'min_samples_split':uniform(),
                  'min_samples_leaf':uniform()}

    # create an instance of the randomized search object
    rsearch = RandomizedSearchCV(DecisionTreeClassifier(random_state=42),
                                 parameters, cv=10, n_iter=100, random_state=42)

    # conduct randomised search over the parameter space
    rsearch.fit(X_train,y_train)
```



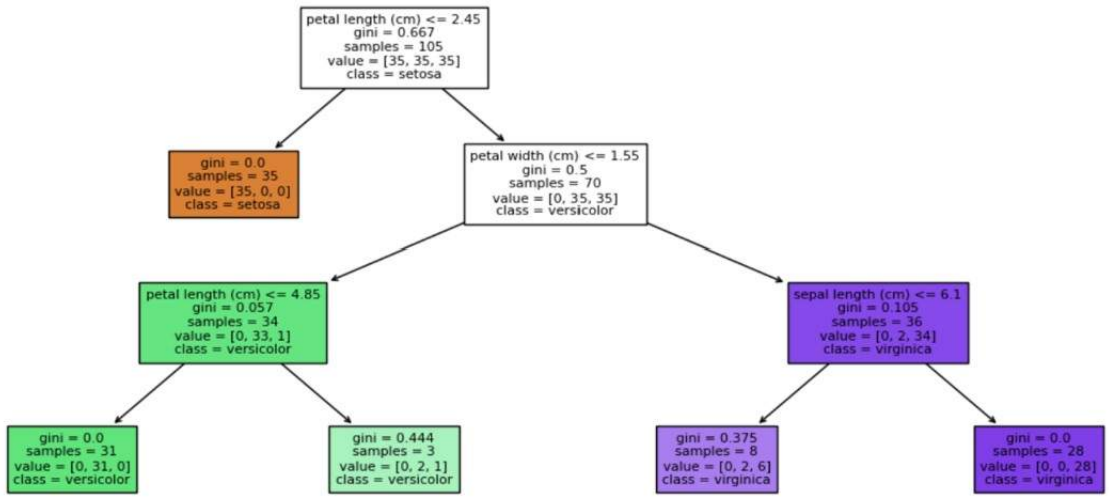
Screenshot 1d from Google Colab by Arden University. (Accessible code).

```
# show best parameter configuration found for classifier
cls_params = rsearch.best_params_
cls_params['min_samples_split'] = np.ceil(cls_params['min_samples_split']*X_train.shape[0])
cls_params['min_samples_leaf'] = np.ceil(cls_params['min_samples_leaf']*X_train.shape[0])
cls_params
```

```
{'max_depth': 3,
 'max_leaf_nodes': 9,
 'min_samples_leaf': 3.0,
 'min_samples_split': 34.0}
```

```
[10] # extract best classifier
clf = rsearch.best_estimator_

# visualise the decision tree
fig = plt.figure(figsize=(14,6))
_ = plot_tree(clf,
              feature_names=feature_names,
              filled=True,
              class_names=class_names,
              fontsize=8)
```

Screenshot 1e from Google Colab by Arden University. (Accessible code).



Screenshot 1f from Google Colab by Arden University. (Accessible code).

```
# produce classification report
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        15
           1       0.86      0.80      0.83        15
           2       0.81      0.87      0.84        15

    accuracy                           0.89        45
   macro avg       0.89      0.89      0.89        45
weighted avg       0.89      0.89      0.89        45
```

```
[12] # step 1: fit a decision tree classifier
     clf = DecisionTreeClassifier(random_state=42)
     clf.fit(X_train,y_train)

     # step 2: extract the set of cost complexity parameter alphas
     ccp_alphas = clf.cost_complexity_pruning_path(X_train,y_train)['ccp_alphas']

     # view the complete list of effective alphas
     ccp_alphas.tolist()
```

```
[0.0,
 0.008994708994708996,
 0.01848739495798319,
 0.27886710239651413,
 0.33333333333333354]
```

Screenshot 1g from Google Colab by Arden University. (Accessible code).

```
# setup parameter space
parameters = {'ccp_alpha':ccp_alphas.tolist()}

# create an instance of the grid search object
gsearch = GridSearchCV(DecisionTreeClassifier(random_state=42), parameters, cv=10)

# step 3: conduct grid search over the parameter space
gsearch.fit(X_train,y_train)
```

```
▸ GridSearchCV
▸ estimator: DecisionTreeClassifier
    ▸ DecisionTreeClassifier
```

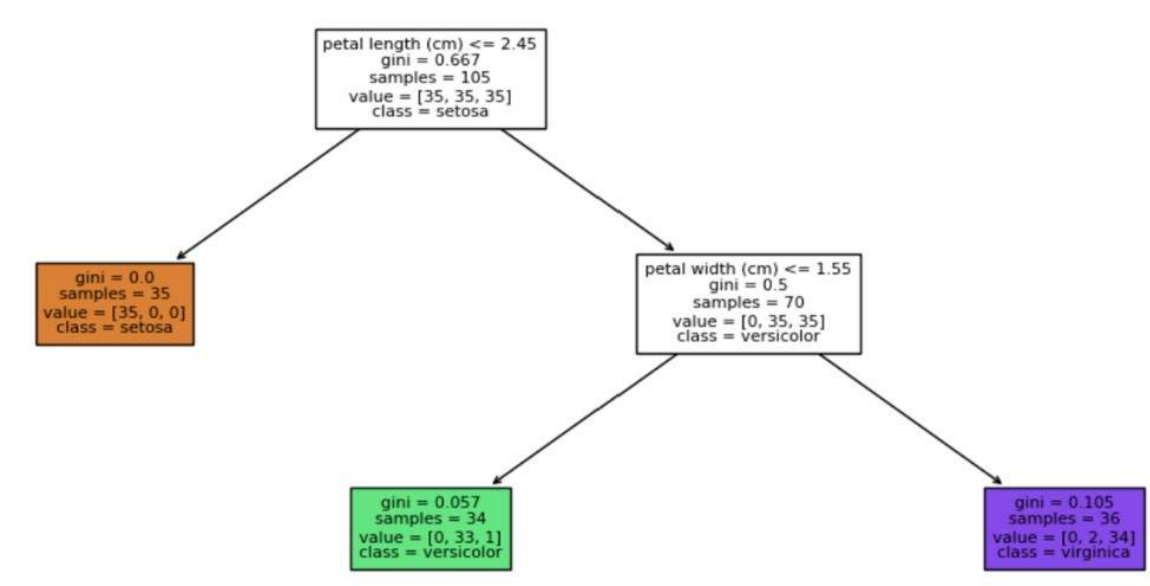
Screenshot 1h from Google Colab by Arden University. (Accessible code).

```
# show best parameter configuration found for classifier
gsearch.best_params_
```

```
{'ccp_alpha': 0.01848739495798319}
```

```
# extract best classifier
clf = gsearch.best_estimator_

# visualise the decision tree
fig = plt.figure(figsize=(14,6))
_ = plot_tree(clf,
              feature_names=feature_names,
              filled=True,
              class_names=class_names,
              fontsize=8)
```

Screenshot 1i from Google Colab by Arden University. (Accessible code).



Screenshot 1j from Google Colab by Arden University.

```
# produce classification report
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        15
           1       0.86      0.80      0.83        15
           2       0.81      0.87      0.84        15

    accuracy                           0.89        45
   macro avg       0.89      0.89      0.89        45
weighted avg       0.89      0.89      0.89        45
```

Screenshot 1k from Google Colab by Arden University. (Accessible code).

Hurrah!

Continue

Topic 4 - Activity 1

Discussion on iLearn

Now that you have fully understood pruning and its benefits in relation to a decision tree, give a full description, in your own words, of the process, what it is used for, why a machine-learning analyst would use the techniques, and the benefits and disadvantages. Write your detailed explanation in the group discussion forum for further discussion with your peers and tutor.

Continue

Topic 4 - Activity 2

Reading

- Chapter 3: Jupyter Notebooks and Google Colab (pp. 53-64) from Python for Scientific Computing and Artificial Intelligence by Lynch
- Chapter 4: Representing Data and Engineering Features (pp. 152-183) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido
- Chapter 4: NumPy Basics: Arrays and Vectorized Computation (pp. 89-121) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney
- Chapter 3: Snakes, Bears & Other Numerical Beasts: NumPy, SciPy & pandas (pp. 137-178) from Statistics and Data Visualisation with Python by Rogel-Salazar

Continue

TOPIC 4 SUMMARY

As we have extensively demonstrated, pruning is a machine-learning technique used to optimise the efficiency of a decision tree; in other words, it limits overfitting within the training data. Moreover, pruning aids in reducing the size of the decision tree by

removing elements that are not really relevant to the model. However, a balance must be sought between complexity and simplicity.

We have now concluded all the topics within this week's module on machine learning.

Continue to Knowledge check

Knowledge check

Select the correct answer for the following questions.

1 of 2: What type of algorithm is a decision tree?

- ◯ Classification
- ◯ Regression
- ◯ Unsupervised

SUBMIT

2 of 2: Why would you use a random forest algorithm over a decision tree?

- ◯ The errors will be fewer, so it will be more accurate
- ◯ There will be more errors, so using the decision tree is better
- ◯ There is no difference in using either

SUBMIT

Continue to Lesson summary

Lesson summary

LESSON SUMMARY

The first topic focused on decision trees. We also demonstrated, both theoretically and numerically, information gain. We initially gave a detailed demonstration of how a decision tree algorithm works and the improved efficiency process involved in producing, as far as possible, the optimum model.

In Topic 2, we looked at an improved machine-learning technique in the information gain process demonstrated in the first topic, namely, gain ratio. Hopefully, Topic 2 has shown you that trade-offs can be expected with a complex process, and gain ratio is no exception.

Topic 3 saw the humble classification tree turned into a forest of trees. Random forest algorithms are widely used within machine learning for obvious reasons. Hopefully, you can now determine why a collection of decision trees is much better than a singular classification decision tree.

In the final topic, we looked at pruning, which is a technique for improving the accuracy and efficiency of a decision tree algorithm.

As we extensively demonstrated, pruning is a machine-learning technique used to optimise the efficiency of a decision tree; in other words, it limits overfitting and underfitting within the training data. Moreover, we also discussed how pruning aids in reducing the size of the decision tree by removing elements that are not really

relevant to the model. However, a balance must be sought between complexity and simplicity.

We have now completed Lesson 5 on supervised machine learning, employing decision trees and their associated processes to improve efficiency and accuracy. See you in the next lesson, which looks at supervised learning using naïve Bayesian classification.

Continue to 'Reading list'

Reading List

All reading and resources in this module are provided by the Arden University Library service. The full reading list for this module can be accessed at the link below. More books, journals and databases can be found on the Library Portal. If you need these resources in a different format, our accessibility tools can help.

Full reading list

You can find all the key, essential and wider reading and resources for this module on the dedicated reading list page on the Learning Success Hub.

VIEW READING LIST

References

Cited sources that do not appear in your reading list:

Department for Digital, Culture, Media & Sport, Office for Artificial Intelligence and Department for Science, Innovation & Technology, (2022). *AI activity in UK businesses: Executive Summary.* GOV.UK [online], 12 January 2022. London: HM Government.

Deutscher, M., (2023). UK to invest £500M more in AI compute capacity, launch five new quantum projects. *SiliconANGLE* [online], 23 November 2023. Palo Alto, CA: SiliconANGLE Media.

LinkedIn, (2023). *16,000+ Machine Learning jobs in United Kingdom* [webpage, online]. Sunnyvale, CA: LinkedIn.

London School of Economics and Political Science, (2023). *10 Tech Careers With The Most Job Security in 2023 | LSE Online* [webpage, online]. London: London School of Economics and Political Science.

Mrkonjić, E., (2022). 25+ Machine Learning Statistics Everyone Should Know. *SeedScientific* [online]. 27 December 2022. New York, NY: SeedScientific.

Parkd, (2018). Artificial intelligence in the mobility sector. *Parkd* [online], 21 September 2018. Antwerp: Parkd.

Talent.com, (2024). *Computer Scientist Salary in United Kingdom - Average Salary* [webpage, online]. Montreal, QC: Talent.com.

Wang, Z., (2022). Survey Shows Contact Center AI Use Is Up Almost Fourfold Since 2019. *CX Today* [online], 18 August 2022. Blackburn: Today Digital.